

SAATHI VYAPAR

साथी व्यापार

COMPLETE A-to-Z VIVA PREPARATION GUIDE

AI-Powered Financial Advisory & Government Scheme Matching
Platform for Rural Micro-Entrepreneurs in India

SIH26091

Ministry of Social Justice & Empowerment

Team Pantheon Eternal

Next.js 16 · React 19 · TypeScript · Supabase · Google Gemini · Tailwind CSS 4

0. How To Use This Document

Ye document project ke **har ek file, har ek function, har ek line ka kaam** cover karta hai, plus viva mein jo bhi pucha ja sakta hai uske ready answers. Structure:

- **Part A (Sec 1-3)** — Problem statement, solution, complete architecture, tech stack (har dependency kyun use hui)
- **Part B (Sec 4-5)** — Folder structure: har file ka exact kaam + saari config files
- **Part C (Sec 6-7)** — Database: har table, har column, RLS security, 15 schemes seed data
- **Part D (Sec 8-9)** — Core engines: Financial Engine + Scheme Matcher, formulas ke saath derivation
- **Part E (Sec 10-12)** — 10 API routes ka complete flow, Conversation Orchestrator state machine, Voice AI
- **Part F (Sec 13-15)** — Auth flow (PKCE), frontend pages, testing
- **Part G (Sec 16-18)** — Deployment, limitations (examiner ye zaroor puchta hai), 120+ Viva Q&A

Viva tip: Sections 8, 9, 10 aur 18 sabse important hain. Agar time kam hai to Section 18 (Q&A Bank) se shuru karo, phir Section 8-9 (engines) dekho — examiner hamesha "core logic" puchta hai.

1. Problem Statement & Solution

1.1 The Problem (Samasya)

India mein **6.3 crore+ micro-enterprises** hain jo unorganized rural commerce ka ~80% chalte hain. Inki 3 core problems:

Problem	Detail	Evidence (project ke andar)
No bookkeeping	Rural entrepreneurs personal aur business ka paisa alag nahi rakhte. Kitna profit hua, pata hi nahi chalta. Sab "gut feel" par chalta hai.	92% undifferentiated cash flow
No advisor	Koi CA ya mentor ₹20,000/month ke business ko time nahi deta — economically unviable hai unke liye.	0 mentor ratio below ₹50K MRR
No scheme visibility	PMEGP, Mudra, Stand-Up India jaisi 100+ schemes exist karti hain, par kisi ko nahi pata ki wo kis ke liye eligible hain.	<4% scheme claim reach

Existing solutions kyun fail hote hain?

- **Jan Samarth / UMANG** — sirf loan discovery karte hain, financial advisory nahi dete.
- Sab **smartphone + app + reading ability** maangte hain — jo target user ke paas nahi hai.
- Koi bhi **voice-first / zero-bandwidth** nahi hai.
- Koi bhi **explainable** nahi hai — bas "eligible/not eligible" bolte hain, reason nahi batate.

1.2 Our Solution — 6 Pillars

Pillar	Kya karta hai	Kaise (technology)
1. Deterministic Financial Engine	Break-even, profit margin, cash-flow risk — sab pure math se. Koi LLM hallucination nahi. Numbers 100% provable.	Pure TypeScript functions, zero dependencies, 100% unit tested
2. Explainable Scheme Matcher	15 real government schemes ke against profile match karta hai aur har scheme ke liye ✓/X reason string deta hai ("Aapki annual revenue ₹5,40,000 scheme limit ₹1,00,00,000 ke andar hai").	Rule-based JSONB eligibility engine
3. Multi-Channel Access	WhatsApp, SMS (feature phone!), aur Web — teeno par same brain chalta hai. SMS matlab internet ki bhi zaroorat nahi.	WhatsApp Cloud API + Twilio + shared orchestrator
4. Voice-First Onboarding	User Hindi/Hinglish mein bol kar apna profile bana sakta hai — padhna-likhna nahi aata to bhi chalega.	Web Speech API (STT + TTS) + Gemini extraction
5. OCR Notebook Digitization	Purani bahi-khata / bill ki photo kheencho, system usme se amounts nikaal kar digital ledger bana deta hai.	Tesseract.js (eng+hin), on-server
6. AI Business Guide	5-stage growth roadmap: Cost Optimization → Value Addition → Direct Market Access → Formal Registration → Scale.	Gemini 2.5 Flash with strict schema + deterministic fallback

1.3 The Core Design Principle (SABSE IMPORTANT ANSWER)

"LLM never touches the numbers."

Saare financial calculations (`break-even` , `margin %` , `risk`) aur saari eligibility decisions **pure deterministic code** se hoti hain. Gemini ka kaam sirf **3 cheezein** hai:

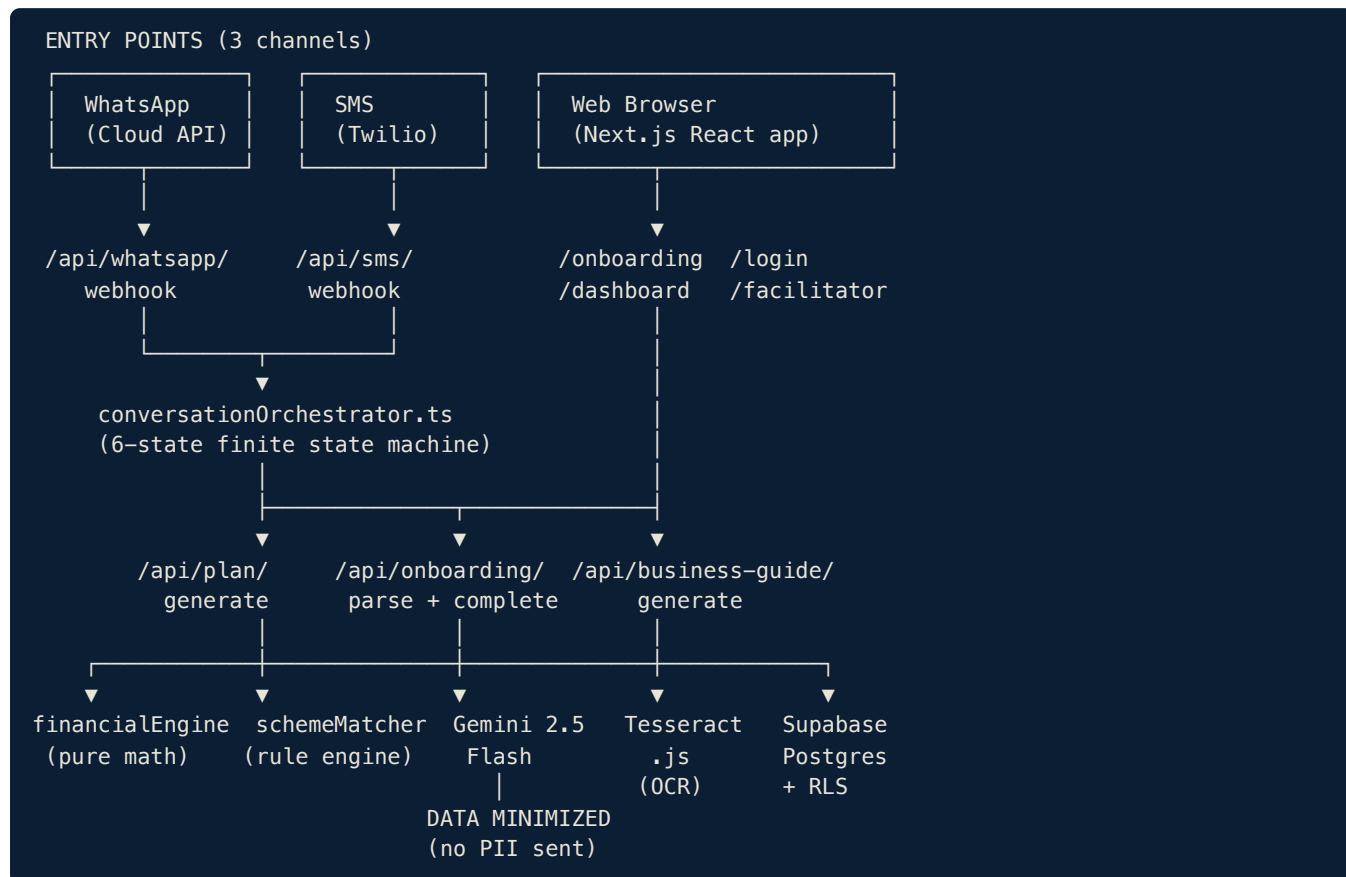
1. **Translate/rephrase** — already-computed numbers ko user ki bhasha mein warm tone mein likhna (`/api/plan/generate`)
2. **Extract** — bola hua vaakya ("pandrah hazaar kamata hoon") se structured field nikalna (`/api/onboarding/parse`)
3. **Advise** — qualitative growth roadmap generate karna (`/api/business-guide/generate`)

Isliye system prompt mein likha hai: *"Do NOT change any numbers — keep them exactly as given."* Aur temperature bhi low rakhi hai (0.1-0.3).

Kyun important hai? Financial advice mein galat number life-changing damage kar sakta hai. Deterministic engine ka output audit/prove kiya ja sakta hai; LLM ka nahi.

2. Complete System Architecture

2.1 Architecture Diagram



2.2 Request Lifecycle — Web par ek user ka poora journey

1. **Landing** (/) — 12-section showcase page. "Try the Demo" par click.
2. **Login** (/login) — Email+Password ya Google OAuth. Supabase Auth handle karta hai.
3. **Callback** (/auth/callback) — PKCE code ko session ke liye exchange karta hai, cookie set karta hai. Agar `business_profiles` row nahi hai → `/onboarding`, warna `/dashboard`.
4. **Onboarding** (/onboarding) — 8-step form ya Voice modal. Har jawab `/api/onboarding/parse` par jata hai (Gemini extraction).
5. **Consent** — Step 8 mein DPDP Act consent. Bina consent ke kuch bhi save nahi hota (Zod `.refine()` se enforce).
6. **Complete** — `/api/onboarding/complete` : user + business_profile save, financial summary compute, schemes match, initial plan insert.
7. **Dashboard** (/dashboard) — Server Component; profile + latest plan + ledger fetch karke metrics dikhata hai. Plan na ho to on-the-fly calculate karta hai.
8. **Yojana Kendra** (/dashboard/schemes) — eligible/ineligible schemes with reasons, required documents checklist, apply links.
9. **Business Guide** (/dashboard/business-guide) — challenge bolo/likho → 5-stage roadmap.

2.3 Request Lifecycle — WhatsApp/SMS par

```
User sends "Hello" to WhatsApp number
↓
Meta hits POST /api/whatsapp/webhook
↓
Route returns 200 IMMEDIATELY (fire-and-forget), processing async
↓
processWhatsAppMessages() extracts phone + text
↓
handleIncomingMessage('whatsapp', '+91...', 'Hello', null)
↓
Orchestrator: upsert user → fetch/create conversation → read state
↓
state = 'idle' → save state='awaiting_sector', return greeting
↓
sendWhatsAppMessage() POSTs reply to graph.facebook.com
```

Interview point: WhatsApp webhook fire-and-forget kyun? Kyunki Meta 20 seconds mein 200 OK expect karta hai, warna message retry karta hai (duplicate messages). Isliye humne `processWhatsAppMessages(body).catch(...)` await nahi kiya — turant 200 return kar diya.

3. Technology Stack — Har Cheez Kyun Use Hui

3.1 Production Dependencies (package.json)

Package	Version	Kaam kya hai / Kyun chuna
next	16.3.4	Full-stack React framework. App Router use kar rahe hain. Ek hi codebase mein frontend (React pages) + backend (API Route Handlers) dono. Server Components se DB directly query kar sakte hain bina separate backend ke. <i>Note: Next.js 16 mein "middleware" ka naam badal kar "proxy" ho gaya hai.</i>
react / react-dom	19.2.8	UI library. React 19 ke Server Components aur <code>useTransition</code> use kiye hain.
@supabase/supabase-js	^2.115	Postgres database + Auth ka JS client. Service-role key ke saath server-side admin access.
@supabase/ssr	^0.12.6	Critical. Session ko <code>localStorage</code> ke bajaye cookies mein store karta hai. Isse Server Components bhi logged-in user ko pehchan sakte hain. Iske bina PKCE OAuth flow tootega.
@google/genai	^2.21	Google Gemini SDK. Model: <code>gemini-2.5-flash</code> . Teen jagah use: plan rephrasing, onboarding parsing, business roadmap.
zod	^4.5.4	Runtime input validation. TypeScript sirf compile-time check karta hai; network se aaya JSON runtime pe kuch bhi ho sakta hai. Zod schema fail hone par 400 + structured error return karta hai.
tesseract.js	^7.0	Pure-JS OCR engine (WebAssembly). <code>'eng+hin'</code> dono language packs load karte hain. Server-side chalata hai — image kabhi kisi third-party OCR service ko nahi bheji jaati (privacy).
twilio	^6.1	SMS channel. Feature phone users ke liye — bina internet ke bhi service.
html-react-parser	^6.1.7	Stitch design panels ke raw HTML strings ko React elements mein convert karta hai (<code>/folio</code> page ke liye).
groq-sdk	^1.6	Installed hai (alternative fast LLM inference ke liye) par current code Gemini use kar raha hai. <i>Viva mein pucha jaye to honestly bolna: "legacy dependency, ab Gemini par migrate kar diya."</i>
supabase (CLI)	^2.116	Migrations push karne ke liye (<code>supabase db push</code>).

3.2 Dev Dependencies

Package	Kaam
<code>typescript ^5</code>	Static typing. <code>strict: true</code> on hai — null safety enforced.
<code>tailwindcss ^4 +</code> <code>@tailwindcss/postcss</code>	Utility-first CSS. v4 mein config JS file ki jagah CSS mein <code>@import "tailwindcss"</code> se hota hai.
<code>vitest ^2.1.9 +</code> <code>@vitest/ui</code>	Unit testing framework (Jest-compatible API, par Vite-fast). 2 test files, ~30 test cases.
<code>eslint ^9 +</code> <code>eslint-config-next</code>	Code linting, Next.js core-web-vitals rules.
<code>@types/*</code>	Type definitions for node, react, react-dom.

3.3 External Services

Service	Use	Env Variable
Supabase	Postgres DB + Auth + RLS	<code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>
Google Gemini	LLM (2.5 Flash)	<code>GEMINI_API_KEY</code>
Meta WhatsApp Cloud API	WhatsApp channel	<code>WHATSAPP_ACCESS_TOKEN</code> , <code>WHATSAPP_PHONE_NUMBER_ID</code> , <code>WHATSAPP_VERIFY_TOKEN</code>
Twilio	SMS channel	<code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_PHONE_NUMBER</code>
Vercel	Hosting / serverless	<code>NEXT_PUBLIC_APP_URL</code>
UptimeRobot	Keepalive pings (Supabase free tier pause hone se bachne ke liye)	—

Security question (zaroor puchega): `NEXT_PUBLIC_` prefix wale variables browser tak jaate hain. Isliye `SUPABASE_SERVICE_ROLE_KEY` par ye prefix **nahi** hai — wo sirf server par rehta hai kyunki wo RLS ko bypass karta hai. Agar wo leak ho jaye to koi bhi poori database padh sakta hai.

4. Complete Folder Structure — Har File Ka Kaam

```
Saathi-Vyapar-main/
├── src/
│   ├── proxy.ts                ← Next.js 16 middleware (session refresh)
│   ├── app/                    ← App Router: pages + API routes
│   │   ├── layout.tsx          ← Root layout, fonts, LanguageProvider
│   │   ├── page.tsx            ← Landing page (12 sections, 1121 lines)
│   │   ├── globals.css         ← Tailwind + custom fonts
│   │   ├── login/
│   │   │   ├── page.tsx        ← Server wrapper + Suspense
│   │   │   ├── LoginForm.tsx   ← Client: sign-in/up, Google OAuth, voice
│   │   │   └── RotatingValueProps.tsx
│   │   ├── onboarding/page.tsx ← 8-step form + voice modal trigger
│   │   ├── dashboard/
│   │   │   ├── page.tsx        ← Server Component: metrics, ledger, charts
│   │   │   ├── LogoutButton.tsx ← Client: supabase.auth.signOut()
│   │   │   ├── schemes/page.tsx ← Yojana Kendra (847 lines)
│   │   │   └── business-guide/page.tsx ← 5-stage roadmap UI
│   │   ├── facilitator/
│   │   │   ├── page.tsx        ← Field worker portal
│   │   │   └── AddEntrepreneurModal.tsx
│   │   ├── folio/page.tsx      ← Design exhibition page
│   │   ├── auth/
│   │   │   ├── callback/route.ts ← OAuth PKCE code exchange
│   │   │   └── confirm/route.ts  ← Email confirmation (3 flows)
│   │   └── api/                ← 8 API route handlers
│   │       ├── plan/generate/route.ts
│   │       ├── onboarding/parse/route.ts
│   │       ├── onboarding/complete/route.ts
│   │       ├── business-guide/generate/route.ts
│   │       ├── ledger/ocr/route.ts
│   │       ├── whatsapp/webhook/route.ts
│   │       ├── sms/webhook/route.ts
│   │       ├── facilitator/add-entrepreneur/route.ts
│   │       └── keepalive/route.ts
│   └── lib/
│       ├── engines/
│       │   ├── financialEngine.ts    ← 166 lines, PURE MATH
│       │   ├── financialEngine.test.ts ← 215 lines of tests
│       │   ├── schemeMatcher.ts      ← 167 lines, RULE ENGINE
│       │   └── schemeMatcher.test.ts  ← 216 lines of tests
│       ├── orchestrator/
│       │   └── conversationOrchestrator.ts ← State machine (378 lines)
│       ├── supabase/
│       │   ├── client.ts             ← Browser client (anon key, RLS ON)
│       │   └── server.ts             ← Server client (service key, RLS OFF)
│       ├── i18n.ts                  ← EN/HI translation dictionary
│       ├── fonts.ts                 ← next/font Google font loaders
│       └── stitchPanelsData.ts      ← 9 design panels data
│   ├── components/
│   │   ├── VoiceOnboardingModal.tsx ← Web Speech API (832 lines)
│   │   ├── LanguageToggleButton.tsx
│   │   └── panels/{StitchFolio,PlatesGallery}.tsx
│   ├── contexts/
│   │   └── LanguageContext.tsx      ← EN/HI React Context
│   └── supabase/
│       ├── migrations/
│       │   └── 001_init.sql         ← 7 tables + indexes
```

```
| | | 002_rls_policies.sql ← Row Level Security
| | | 004_business_guides.sql ← 8th table
| | seed/schemes.sql      ← 15 real govt schemes
| | schema.sql            ← Full consolidated schema
| | README.md             ← Migration instructions
| public/                 ← Logo, fonts, panel images
| stitch-panels/html/     ← 9 raw design HTML panels
| graphify-out/           ← Code dependency graph (analysis tool output)
| package.json, tsconfig.json, next.config.ts, vitest.config.ts,
| eslint.config.mjs, postcss.config.mjs, vercel.json
| schema.sql, design.md, README.md, AGENTS.md, CLAUDE.md
```

4.1 File-by-File Responsibility Table

File	Lines	Exact Responsibility
CORE LOGIC (Business Brain)		
lib/engines/financialEngine.ts	166	4 pure functions: <code>calculateBreakEven</code> , <code>calculateMarginPercent</code> , <code>assessCashFlowRisk</code> , <code>generateFinancialSummary</code> . Zero imports — completely dependency-free, hence 100% testable.
lib/engines/schemeMatcher.ts	167	<code>matchSchemes(profile, schemes[])</code> — 6 rule checks, returns eligible flag + human-readable ✓/✗ reasons, sorted eligible-first.
lib/orchestrator/conversationOrchestrator.ts	378	6-state finite state machine for WhatsApp/SMS. User upsert, conversation persistence, amount/yes-no parsing, bilingual message dictionary.
DATA ACCESS		
lib/supabase/client.ts	33	<code>createBrowserClient</code> with ANON key. Cookie-based session. RLS policies apply. Safe for browser.
lib/supabase/server.ts	63	Two exports: <code>supabaseServer</code> (SERVICE_ROLE key, bypasses RLS, server-only) and <code>createSupabaseRouteClient()</code> (per-request, cookie-bound, respects RLS — use jab "kaun logged in hai" jaanna ho).
proxy.ts	55	Runs on every non-static request. Calls <code>supabase.auth.getUser()</code> which silently refreshes an expired access token and rewrites the cookie. Wrapped in try/catch so an Auth outage never 500s the page.
API ROUTES		
api/plan/generate	269	POST. Zod-validate <code>user_id</code> → fetch profile → compute metrics → match schemes → Gemini rephrase (PII-free) → save to <code>financial_plans</code> → return plan.
api/onboarding/parse	240	POST. Step + transcript → Gemini structured JSON extraction. Regex fallback parser agar Gemini key missing ho ya call fail ho.
api/onboarding/complete	245	POST. DPDP consent enforce → user create/upsert → <code>business_profile</code> upsert → compute plan → save. Returns redirect URL.
api/business-guide/generate	286	POST. 5 FIXED stages enforce karta hai. Gemini output ko validate karta hai (length===5), warna Hindi deterministic fallback roadmap.
api/ledger/ocr	192	POST multipart. Tesseract 'eng+hin' → 3 regex patterns se amounts nikalta hai →

`ledger_entries` mein `confirmed:false` ke saath insert.

<code>api/whatsapp/webhook</code>	185	GET = Meta verification (hub.challenge echo). POST = messages → orchestrator → reply via Graph API.
<code>api/sms/webhook</code>	77	POST form-urlencoded → orchestrator → TwiML XML response (XML-escaped to prevent injection).
<code>api/facilitator/add-entrepreneur</code>	106	POST. Phone normalize (+91) → user find/create → profile upsert → link row in <code>facilitators_entrepreneurs</code> .
<code>api/keepalive</code>	54	GET. DB connectivity check + scheme count. <code>Cache-Control: no-store</code> . UptimeRobot isko ping karta hai.
<code>auth/callback/route.ts</code>	107	OAuth PKCE code → session exchange. Post-login smart routing: profile hai to dashboard, nahi to onboarding.
<code>auth/confirm/route.ts</code>	96	Email confirmation — 3 fallback flows: token_hash OTP, PKCE code, legacy token+email OTP.

UI PAGES

<code>app/page.tsx</code>	1121	Landing page: nav, hero, trust strip, glassmorphic collage, manifesto, 2 feature splits, use-case grid with persona filter, testimonial, roadmap, CTA, footer.
<code>app/dashboard/page.tsx</code>	686	Server Component (<code>dynamic='force-dynamic'</code>). 4-level user resolution fallback chain, profile fetch, plan fetch (or live-compute), 15-day ledger chart, scheme cards.
<code>app/dashboard/schemes/page.tsx</code>	847	Client. Yojana Kendra: eligible/ineligible tabs, per-scheme sponsoring ministry + document checklist (<code>getSchemeDetails()</code>), "Recheck" button with <code>useTransition</code> .
<code>app/onboarding/page.tsx</code>	756	8-step wizard with progress bar. Voice modal auto-opens if browser supports Speech API; text form is the accessible fallback.
<code>components/VoiceOnboardingModal.tsx</code>	832	Web Speech API: <code>SpeechRecognition</code> (lang <code>hi-IN</code> , interim results) + <code>speechSynthesis</code> TTS. 9-step conversational flow.
<code>app/facilitator/page.tsx</code>	241	Server Component. Lists all entrepreneurs with their profile + latest plan. Field-worker view.

5. Configuration Files Explained

5.1 next.config.ts

```
const nextConfig: NextConfig = {
  async redirects() {
    return [
      { source: '/signup', destination: '/login', permanent: false },
      { source: '/register', destination: '/login', permanent: false },
    ];
  },
};
```

Kaam: `/signup` aur `/register` URLs ko `/login` par bhejta hai kyunki login page hi sign-in aur sign-up dono handle karta hai (tabs ke through). `permanent: false` = HTTP 307 (temporary) — 308 nahi, taaki future mein alag signup page banane par browsers ne cache na kiya ho.

5.2 tsconfig.json — key options

Option	Matlab
<code>"strict": true</code>	Null/undefined checks, implicit any ban. Sabse important type-safety flag.
<code>"paths": {"@/*": [".src/*"]}</code>	Path alias — <code>import from '@lib/engines/...'</code> likh sakte hain, <code>../../../../lib</code> nahi. Vitest config mein bhi same alias duplicate kiya hai warna tests break honge.
<code>"moduleResolution": "bundler"</code>	Modern resolution — Next.js/Turbopack ke liye.
<code>"jsx": "react-jsx"</code>	React 17+ automatic JSX transform — har file mein <code>import React</code> zaroori nahi.
<code>"noEmit": true</code>	TS sirf type-check karta hai; actual compilation Next.js/SWC karta hai.

5.3 vitest.config.ts

```
test: {
  globals: true, // describe/it/expect bina import ke
  environment: 'node', // jsdom nahi — engines pure JS hain
  include: ['src/**/*.test.ts(x)'],
  coverage: { provider: 'v8', include: ['src/lib/**/*.ts'] },
},
resolve: { alias: { '@': path.resolve(__dirname, './src') } }
```

Note: `environment: 'node'` chuna kyunki humare tests sirf pure functions test karte hain — koi DOM nahi chahiye. Ye tests ko bahut fast banata hai.

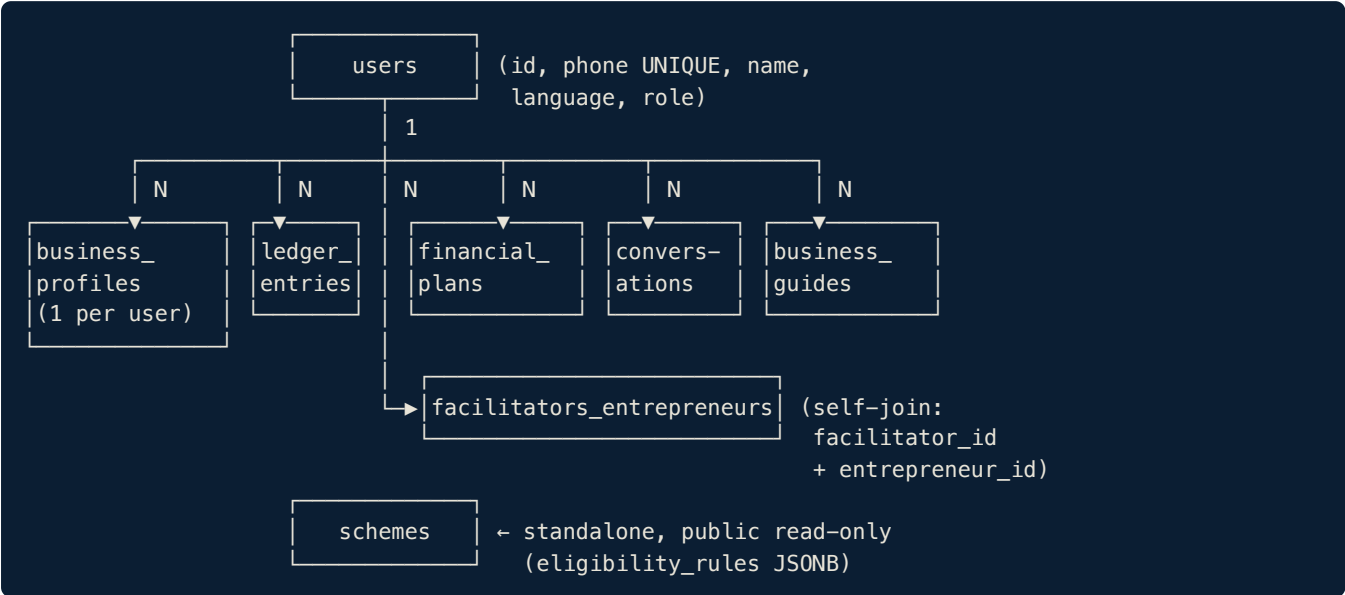
5.4 vercel.json / postcss.config.mjs / eslint.config.mjs

- `vercel.json` — sirf `{"framework": "nextjs"}`. Vercel ko batata hai build command auto-detect karna hai.
- `postcss.config.mjs` — Tailwind v4 ka PostCSS plugin register karta hai.

- `eslint.config.mjs` — Flat config format (ESLint 9). next core-web-vitals + TypeScript rules. `no-page-custom-font` rule off kiya hai kyunki humne `layout.tsx` ke `<head>` mein directly Google Fonts link kiya hai.
- `.gitignore` — `.env*` ignore hai (API keys leak na ho), aur AI instruction files (`AGENTS.md` , `CLAUDE.md`) bhi.

6. Database Design (Supabase Postgres)

6.1 Entity Relationship



6.2 Table-by-Table Breakdown

1. users

Column	Type	Purpose
id	UUID PK, default <code>uuid_generate_v4()</code>	Primary key. Supabase Auth ke <code>auth.users.id</code> se match karaya jata hai jab web se signup hota hai.
phone	VARCHAR(20) UNIQUE NOT NULL	WhatsApp/SMS ke liye identity. E.164 format (+919876543210).
name	VARCHAR(255)	Display name.
language	VARCHAR(10) default 'hi'	Reply language. Default Hindi kyunki target rural users.
role	VARCHAR(20) CHECK IN ('entrepreneur', 'facilitator', 'admin')	RBAC. CHECK constraint DB level par galat value rokta hai.
created_at / updated_at	TIMESTAMPTZ default NOW()	Audit trail. TIMESTAMPTZ (timezone-aware) use kiya, plain TIMESTAMP nahi.

2. business_profiles

Ek user ka ek business profile. `user_id` par `ON DELETE CASCADE` — user delete hua to profile bhi.

Column	Purpose
business_name, sector, district, state	Business identity aur location. <code>sector</code> scheme matching ka key input hai.
monthly_revenue_est / monthly_expense_est	<code>NUMERIC(12,2)</code> — 12 total digits, 2 decimal. <code>FLOAT</code> nahi use kiya kyunki paise mein floating-point rounding errors nahi chahiye.
existing_loans	<code>BOOLEAN</code> . Risk assessment mein directly use hota hai.
category	'sc' / 'st' / 'obc' / 'general' / 'minority' — scheme eligibility ke liye.
gender	<code>CHECK IN ('male','female','other')</code> . Mahila-specific schemes ke liye.

Real bug jo fix hua (great story for viva): Onboarding form gender collect nahi karta tha, to code `gender: 'any'` default bhej raha tha. Par DB par `CHECK (gender IN ('male', 'female', 'other'))` constraint tha — har submission "Failed to save business profile" se fail ho rahi thi. Fix: default `'other'` kar diya. Ye batata hai ki DB constraints application bugs pakadte hain.

3. schemes

Column	Purpose
name, description, benefit_summary	Scheme ki text info.
eligibility_rules JSONB	Design ka dil. Har scheme ke rules JSON mein: <code>{income_max, category[], sector[], gender, state, loan_amount_min, loan_amount_max}</code> . JSONB isliye taaki nayi scheme add karne ke liye schema migration na karni pade — bas ek <code>INSERT</code> .
application_link	Official government portal URL.

4. financial_plans

Har generated plan ka snapshot — historical record. Columns: `break_even_units`, `margin_percent NUMERIC(5,2)`, `plan_json JSONB` (full detail), `matched_scheme_ids UUID[]` (Postgres array type), `summary_text` (LLM-rephrased text).

Kyun dono `matched_scheme_ids` array aur `plan_json` ? Array fast querying ke liye ("kitne users PMEGP ke eligible hain?"), JSONB complete audit trail ke liye (reasons, timestamps). Ye intentional denormalization hai.

5. ledger_entries

Column	Purpose
amount NUMERIC(12,2) NOT NULL	Transaction amount.
entry_type CHECK IN ('income','expense')	Direction.
source CHECK IN ('manual','ocr','whatsapp')	Provenance tracking — pata rahe ki entry kahan se aayi.
confirmed BOOLEAN default FALSE	Human-in-the-loop safety. OCR se aayi entries <code>confirmed=false</code> hoti hain. User dashboard par confirm karta hai tabhi wo real financial data banti hai. OCR galat padh sakta hai — isliye auto-trust nahi.

6. conversations

`channel` ('whatsapp','sms'), `state` (FSM current state), `context` JSONB (ab tak collect kiya data), `last_message_at`. **Ye table stateless serverless ko stateful banata hai** — har webhook call ek naya function instance hai, memory nahi hoti, isliye state DB mein persist karna zaroori hai.

7. facilitators_entrepreneurs

Many-to-many self-referencing junction table. Dono FKs `users(id)` par. `UNIQUE(facilitator_id, entrepreneur_id)` duplicate links rokta hai — aur yahi `onConflict` target hai upsert ke liye.

8. business_guides (migration 004)

`input_text` (user ka challenge), `roadmap_json` JSONB (5-stage array). Indexes: `user_id` aur `created_at` DESC (latest-first queries fast).

6.3 Indexes — Kyun Banaye

Index	Kyun
<code>idx_users_phone</code>	Har WhatsApp/SMS message phone se user dhundta hai — sabse frequent lookup.
<code>idx_business_profiles_user_id</code> , <code>idx_financial_plans_user_id</code> , <code>idx_ledger_entries_user_id</code> , <code>idx_conversations_user_id</code>	Har FK par index — warna har join full table scan karega.
<code>idx_ledger_entries_created_at</code>	Dashboard 15-din ka chart banata hai — date-range query fast chahiye.

Sab `CREATE ... IF NOT EXISTS` hain — migrations **idempotent** hain, dobara chalane par error nahi.

7. Security — Row Level Security (RLS)

7.1 RLS kya hai?

Postgres ka feature: har table par policies lagti hain jo **database level** par decide karti hain ki kaunsi rows kaun dekh/badal sakta hai. Supabase har request ke JWT se `auth.uid()` nikaalta hai. Matlab agar application code mein bug bhi ho, tab bhi user doosre ka data nahi padh sakta — **defense in depth**.

7.2 Humari Policies (002_rls_policies.sql)

Table	Operation	Policy Logic
schemes	SELECT	USING (true) for authenticated, anon — public data hai, sabko padhne do. Koi INSERT/UPDATE policy nahi = koi user scheme add/edit nahi kar sakta (sirf admin service-role se).
users	SELECT	<code>auth.uid() = id OR id IN (SELECT entrepreneur_id FROM facilitators_entrepreneurs WHERE facilitator_id = auth.uid())</code> — apna profile ya apne linked entrepreneurs.
users	UPDATE / INSERT	Sirf <code>auth.uid() = id</code> . Facilitator dekh sakta hai par edit nahi — read-only supervision.
business_profiles, financial_plans, ledger_entries, business_guides	SELECT	Same dual pattern: own data OR facilitator-linked.
business_profiles, ledger_entries, business_guides	INSERT/UPDATE/DELETE	Sirf owner (<code>auth.uid() = user_id</code>).
conversations	ALL	Sirf owner — facilitator ko bhi chat history nahi dikhti (privacy).
facilitators_entrepreneurs	SELECT	Dono sides dekh sakte hain (<code>auth.uid() = facilitator_id OR = entrepreneur_id</code>) — entrepreneur ko pata hona chahiye ki kaun uska data dekh raha hai (transparency).
facilitators_entrepreneurs	INSERT/DELETE	Sirf facilitator.

7.3 USING vs WITH CHECK

USING = existing rows par filter (SELECT, UPDATE, DELETE ke liye) — "kaunsi rows dikhengi".

WITH CHECK = new/modified row par validation (INSERT, UPDATE ke liye) — "kaunsi row likhi ja sakti hai".

UPDATE par **dono** chahiye: **USING** se decide ki kaunsi row edit kar sakte ho, **WITH CHECK** se rokta hai ki edit karke `user_id` kisi aur ka na bana do (ownership hijack attack).

7.4 Service Role Key – RLS Bypass

Humare API routes `supabaseServer` (service-role key) use karte hain jo RLS **bypass** karta hai. Kyun zaroori?

- WhatsApp/SMS user ke paas koi JWT session nahi hai — sirf phone number hai. RLS ke through wo kuch nahi kar sakta.
- Webhook processing server-to-server hai, browser session nahi.

Trade-off: Isse security boundary application code par shift ho jati hai. *Honest limitation batao viva mein.*

Mitigation: key kabhi client par nahi jati (`NEXT_PUBLIC_` prefix nahi), aur `persistSession: false`, `autoRefreshToken: false` set kiya hai taaki wo client ki tarah behave na kare.

7.5 15 Government Schemes (seed/schemes.sql)

#	Scheme	Benefit	Key Eligibility Rules
1	PMEGP	15-35% subsidy, max ₹25L manufacturing	income_max 1Cr; sectors: manufacturing, services, retail, food_processing, handicrafts, textile
2	Mudra Shishu (PMMY)	Collateral-free upto ₹50,000	income_max ₹25L; loan 1,000-50,000
3	Mudra Kishor (PMMY)	₹50,001 - ₹5 lakh	income_max ₹50L
4	Mudra Tarun (PMMY)	₹5L - ₹10 lakh	income_max ₹1.5Cr
5	Stand-Up India	₹10L - ₹1 crore greenfield	category: SC/ST; sectors: manufacturing, services, trading
6	PM SVANidhi	₹10k→₹20k→₹50k street vendor credit	Open scheme, broad rules
7	PM-AJAY	SC community skill + livelihood	category: SC
8	NSFDC	Term loans @ 6% p.a.	category: SC, below 2x poverty line
9	NSTFDC	Loans @ 6-8%	category: ST
10	Mahila Samridhi Yojana	₹1.4L micro-credit @ 4%	gender: female + category: SC
11	Udyam Sakhi	Training + mentorship + credit	gender: female
12	PMKVY (Skill India)	Free training, ₹8,000 reward	Broad
13	DAY-NRLM	SHG loans @ 7% for women	Rural, SHG-linked
14	CGTMSE	75-85% credit guarantee upto ₹5Cr	sectors incl. technology; loan upto ₹5Cr
15	KVIC Honey Mission	Free training + 50 beehives	sectors: agriculture, beekeeping; income_max ₹12L

Note: Ministry of Social Justice & Empowerment ki schemes (PM-AJAY, NSFDC, NSTFDC, Mahila Samridhi) deliberately include ki hain kyunki SIH problem statement usi ministry ka hai.

8. Financial Engine — Line by Line

src/lib/engines/financialEngine.ts — 166 lines, **zero imports**. Ye project ka sabse important file hai.

8.1 Function 1: calculateBreakEven()

```
export function calculateBreakEven(  
  fixedCosts: number, pricePerUnit: number, variableCostPerUnit: number  
): number {  
  const contributionMargin = pricePerUnit - variableCostPerUnit;  
  if (contributionMargin <= 0) return Infinity; // GUARD  
  return fixedCosts / contributionMargin;  
}
```

Formula & Derivation

Break-Even point wo hai jahan **Total Revenue = Total Cost**:

$$P \times Q = FC + (VC \times Q)$$

$$P \times Q - VC \times Q = FC$$

$$Q \times (P - VC) = FC$$

$$Q = FC / (P - VC)$$

Jahan: P = price per unit, VC = variable cost per unit, FC = fixed costs, Q = break-even quantity.

(P – VC) ko **Contribution Margin** kehte hain — har unit bechne par jo paisa fixed costs cover karne mein "contribute" karta hai.

Example

Ek tailor: dukan ka kiraya + bijli = ₹10,000/month (fixed). Ek shirt ₹500 mein bechta hai (price), kapda+dhaga ₹300 lagta hai (variable).

Contribution Margin = 500 – 300 = ₹200

Break-even = 10,000 / 200 = **50 shirts per month**. 50 shirt bechne par profit zero, 51st shirt se profit shuru.

Edge Cases (Examiner yahi puchta hai!)

Case	Return	Kyun
$P = VC$ (margin = 0)	Infinity	Har unit par zero contribution. Chahe crore units becho, fixed cost kabhi cover nahi hogi. Mathematically $x/0 = \text{Infinity}$. Crash nahi karta — JS mein <code>10000/0</code> Infinity deta hai, par humne explicitly guard lagaya taaki intent clear rahe.
$P < VC$ (negative margin)	Infinity	Har sale par loss ho raha hai. Zyada bechoge to zyada doobo ge. Negative number return karna misleading hota.
$FC = 0$, margin > 0	0	Koi fixed cost nahi = pehli sale se hi profit. Correct.
$FC = 0$, margin = 0	Infinity	Guard pehle chalta hai. $0/0 = \text{NaN}$ hota, jo <code>isFinite()</code> checks tod deta — isliye guard pehle.

8.2 Function 2: calculateMarginPercent()

```
export function calculateMarginPercent(revenue: number, expenses: number): number {
  if (revenue === 0) return 0; // GUARD: div by zero
  return ((revenue - expenses) / revenue) * 100;
}
```

Formula: $\text{Margin\%} = (\text{Revenue} - \text{Expenses}) / \text{Revenue} \times 100$

Input	Output	Meaning
(10000, 7000)	30	30% profit margin — healthy
(5000, 8000)	-60	Loss. Negative allow karte hain — clamp nahi karte, kyunki loss ka magnitude batana zaroori hai.
(5000, 5000)	0	Exact break-even
(0, 5000)	0	Guard. Mathematically $-\text{Infinity}$ hota, par $-\text{Infinity}\%$ margin user ko dikhana bekaar hai.
(10000, 0)	100	Koi kharcha nahi = 100% margin

Question: revenue=0 par 0 return karna misleading nahi hai?

Answer: Haan, thoda hai. Par is case ko `assessCashFlowRisk()` handle karta hai — wo revenue=0 par turant `'high'` risk return karta hai. Do functions milkar poori picture dete hain: margin 0% + risk HIGH = user ko clear signal.

8.3 Function 3: assessCashFlowRisk()

```
export function assessCashFlowRisk(
  monthlyRevenue: number, monthlyExpenses: number, existingLoans: boolean
): 'low' | 'medium' | 'high' {
  if (monthlyRevenue === 0) return 'high'; // Rule 0
  const ratio = monthlyExpenses / monthlyRevenue; // expense ratio
  if (ratio >= 0.9 || (existingLoans && ratio >= 0.7)) return 'high';
  if (ratio >= 0.7 || existingLoans) return 'medium';
  return 'low';
}
```

Decision Table (order matters!)

Priority	Condition	Result	Business reasoning
0	revenue == 0	HIGH	Koi income nahi = maximum distress. Ratio calculate karna bhi possible nahi ($\div 0$).
1	ratio ≥ 0.9	HIGH	90%+ income kharch ho rahi hai. Koi buffer nahi — ek bhi emergency (bimari, machine kharab) business band kara degi.
2	loans AND ratio ≥ 0.7	HIGH	Compound risk. 70% kharch + EMI ka bojh = default ka khatra. Loan ratio threshold ko 0.9 se 0.7 tak neeche le aata hai.
3	ratio ≥ 0.7	MEDIUM	Buffer patla hai par manageable.
4	existingLoans (koi bhi ratio)	MEDIUM	Loan hone se kabhi 'low' nahi mil sakta — fixed obligation hamesha risk add karta hai.
5	default	LOW	<70% kharch, koi loan nahi = healthy.

VERY LIKELY QUESTION: "0.7 aur 0.9 thresholds kahan se aaye?"

Answer: Ye microfinance industry ki common practice se liye gaye heuristics hain — 30% surplus ko healthy working-capital buffer mana jata hai, aur 10% se kam surplus ko distress. Ye **tunable constants** hain, hardcoded magic numbers nahi — future mein sector-wise calibrate kiye ja sakte hain (dairy ka margin structure retail se alag hota hai). Filhaal MVP ke liye conservative aur explainable rakhe hain.

8.4 Function 4: generateFinancialSummary() — Orchestrator

Teeno functions ko call karke ek complete summary + **template-based English explanation** banata hai.

```
const breakEvenUnits = calculateBreakEven(
  monthlyExpenseEst, // fixedCosts = monthly expenses
  monthlyRevenueEst, // price proxy
  0, // variable cost unknown at onboarding
);
```

Ye sabse tricky part hai — honest rehna:

Onboarding par humein **per-unit data nahi milta** (ek shirt kitne ki, ek shirt ka material kitna). User sirf total monthly revenue aur expense batata hai. To true unit-based break-even calculate nahi ho sakta.

Approximation: `fixedCosts = monthly expenses`, `price = monthly revenue`, `variableCost = 0` daal diya. Result = `expenses/revenue` ka ratio, jo units nahi balki **"revenue coverage multiple"** hai.

Isliye explanation string mein humne ise units ki tarah dikhaya hi nahi — bolte hain: *"You need to maintain at least ₹{expenses} in monthly revenue to cover your costs."* Yani user ko **revenue target** batate hain, fake unit count nahi.

Ye code ke comment mein bhi explicitly documented hai — future improvement: onboarding mein per-unit price/cost poochna.

Explanation string kaise banti hai

4 pieces concatenate hote hain:

1. **Risk description** — `riskDescriptions[cashFlowRisk]` lookup object se (low/medium/high ke liye alag warm paragraph)
2. **Loan note** — agar loan hai to EMI warning add hota hai
3. **Margin note** — positive: "Your current profit margin is X%", negative: "You are operating at a loss of X%" (`Math.abs()` se sign hata kar)
4. **Break-even note** — Infinity ho to "cannot be calculated", warna revenue target

Return mein `parseFloat(x.toFixed(2))` use hua hai — 2 decimal par round karke wapas number banata hai (string nahi).

8.5 Testing (financialEngine.test.ts — 215 lines)

Test Group	Cases	Kya cover hota hai
calculateBreakEven	7	Normal, zero margin, negative margin, FC=0 dono cases, large numbers (1 crore), fractional (<code>toBeCloseTo</code>)
calculateMarginPercent	7	Positive, negative (loss), exact zero, revenue=0, expenses=0, tiny margin, both zero
assessCashFlowRisk	9	Boundary testing: ratio 0.69 (low), exactly 0.7 (medium), 0.75, exactly 0.9 (high), loans+0.7 (high), revenue=0
generateFinancialSummary	~5	Structure validation (saare 4 properties), healthy business, stressed business

Boundary Value Analysis highlight karo viva mein: humne exactly 0.69, 0.70, 0.90 test kiye hain — kyunki off-by-one/comparison bugs (`>` vs `>=`) hamesha boundaries par milte hain. Ye professional testing practice hai.

Run karne ka command: `npx vitest run` | Coverage: `npx vitest run --coverage` (v8 provider)

9. Scheme Matcher Engine — Line by Line

src/lib/engines/schemeMatcher.ts — 167 lines, pure TypeScript.

9.1 Signature & Data Shapes

```
matchSchemes(profile: BusinessProfile, schemes: SchemeRecord[]): MatchResult[]

interface MatchResult {
  scheme: SchemeRecord;
  eligible: boolean;
  reasons: string[]; // human-readable ✓ / ✗ strings
}
```

Algorithm: `schemes.map()` → har scheme par 6 checks → phir `sort()` (eligible pehle). **Complexity: $O(n \log n)$** jahan n = schemes count (sort dominate karta hai; 15 schemes = trivial).

9.2 The 6 Rule Checks

Check 1 — Income / Revenue

```
if (rules.income_max !== undefined) {
  const annualRevenue = profile.monthly_revenue_est * 12;
  if (annualRevenue <= rules.income_max) {
    reasons.push(`✓ Your annual revenue (₹${annualRevenue.toLocaleString('en-IN')})
      is within the scheme limit (₹...)`);
  } else { eligible = false; reasons.push('✗ ... exceeds the income limit ...'); }
}
```

- **monthly × 12** — schemes annual income limits mein defined hain, humara data monthly hai.
- **toLocaleString('en-IN')** — Indian numbering system! 5,40,000 (lakh format), na ki 540,000 (western). Chhoti detail, par rural user ke liye readability mein bada farak.
- **!== undefined** check kyun? Kyunki `income_max: 0` valid value ho sakti hai; simple truthy check (`if (rules.income_max)`) usko skip kar deta.

Check 2 — Social Category (SC/ST/OBC/General/Minority)

```
if (rules.category && rules.category.length > 0) {
  const profileCategory = profile.category?.toLowerCase();
  if (profileCategory &&
    rules.category.map(c => c.toLowerCase()).includes(profileCategory)) { ✓ }
  else { eligible = false; ✗ }
}
```

Dono sides lowercase kiye — "SC" vs "sc" vs "Sc" ka mismatch na ho. Agar user ne category nahi di, to `'not specified'` dikhate hain reason mein (crash nahi).

Check 3 — Sector

Same array-includes pattern. Scheme `sector: ["manufacturing","services"]` define karti hai; profile ka sector usme hona chahiye.

Check 4 — Gender

```
if (rules.gender && rules.gender !== 'any') { ... }
```

'any' ko explicitly skip karte hain — wo "no restriction" ka sentinel value hai.

Check 5 — State Restriction

Case-insensitive exact string match. *Limitation*: "UP" vs "Uttar Pradesh" vs "उत्तर प्रदेश" match nahi hoga — future mein normalization map chahiye.

Check 6 — Empty Rules = Universal

```
if (Object.keys(rules).length === 0) {  
  reasons.push('✓ This scheme has no specific eligibility restrictions – broadly applicable');  
}
```

Check 7 — Safety Net

```
if (eligible && reasons.length === 0) {  
  reasons.push('✓ You appear to meet all eligibility criteria for this scheme');  
}
```

Defensive: UI kabhi khali reasons list na dikhaye.

9.3 Sorting

```
return results.sort((a, b) => {  
  if (a.eligible === b.eligible) return 0; // stable: original order preserved  
  return a.eligible ? -1 : 1; // eligible first  
});
```

`return 0` equal case mein — ES2019+ mein `Array.sort` **stable** hai, to same-eligibility schemes apna DB order maintain karti hain (predictable UI).

9.4 Design Philosophy — 4 Key Decisions

Decision	Rationale
AND logic (sab rules pass hone chahiye)	Government eligibility conjunctive hoti hai. Ek bhi criterion fail = ineligible. Legally accurate.
Reasons ineligible schemes ke liye bhi collect karte hain	Loop early-exit nahi karta jab <code>eligible=false</code> ho jata hai. Kyunki user ko batana hai ki "aap kyun eligible nahi ho" — shayad wo gap close kar sake (jaise caste certificate banwa le). Ye "Explainable AI" ka core hai.
JSONB rules, hardcoded if-else nahi	Nayi scheme add karni ho to sirf ek SQL INSERT — code redeploy nahi. Non-technical person bhi Supabase UI se kar sakta hai.
Pure function, no side effects	Same input = same output, hamesha. DB call nahi, network call nahi, <code>Date.now()</code> nahi. Isliye 100% testable aur debuggable.

9.5 Test Coverage (schemeMatcher.test.ts — 8 scenarios)

1. SC entrepreneur with low revenue → PMEGP + Stand-Up India match
2. Female ST entrepreneur → Mahila Samridhi + Stand-Up India
3. High-revenue general male → PMEGP income limit exceed (negative test)
4. Agriculture sector → KVIC Honey Mission match
5. Any entrepreneur → PM SVANidhi (open scheme)
6. Sorting: eligible schemes ineligible se pehle aati hain
7. Reasons mein ✓ checkmark characters present hain
8. No category → category-restricted schemes fail hoti hain

Notice: positive aur negative dono cases test kiye hain. "Sab pass ho gaya" wala test bekaar hota hai — rejection bhi verify karna zaroori hai.

9.6 Worked Example (viva mein ye banake dikha sakte ho)

Profile: Sunita, female, SC category, tailoring (sector: manufacturing), Bihar, ₹18,000/month revenue, ₹11,000 expense, no loans.

Annual revenue = $18,000 \times 12 = ₹2,16,000$

Mahila Samridhi Yojana (rules: gender=female, category=[sc]):

✓ gender female matches

✓ category 'sc' is in ['sc']

→ **ELIGIBLE**

Stand-Up India (rules: category=[sc,st], sector=[manufacturing,services,trading]):

✓ category sc in [sc, st]

✓ sector manufacturing in list

→ **ELIGIBLE**

KVIC Honey Mission (rules: sector=[agriculture, beekeeping, rural_livelihood], income_max=12,00,000):

✓ $2,16,000 \leq 12,00,000$

✗ sector 'manufacturing' is not in [agriculture, beekeeping, rural_livelihood]

→ **NOT ELIGIBLE** (par reason user ko dikhta hai)

Financial metrics: margin = $(18000 - 11000) / 18000 \times 100 = 38.89\%$

ratio = $11000 / 18000 = 0.611 \rightarrow < 0.7$ aur no loans → **risk = LOW**

10. API Routes — Complete Flow of Each

10.1 POST /api/plan/generate Core

Input: `{ user_id: UUID }` | 8-step flow:

1. **Parse JSON** — try/catch, malformed body par 400 "Invalid JSON"
2. **Zod validate** — `z.object({ user_id: z.string().uuid() }). Fail par 400 + parsed.error.flatten() (structured field-wise errors)`
3. **Fetch user** — language preference ke liye. Na mile to 404
4. **Fetch business profile** — latest wala (`order created_at desc, limit 1`). Na mile to 404 "complete onboarding first"
5. **Compute metrics** — `generateFinancialSummary()` call
6. **Fetch + match schemes** — saari schemes laao, `matchSchemes()` chalao, eligible filter karo
7. **Gemini rephrase** — **PII-FREE** (neech detail)
8. **Save + return** — `financial_plans` mein insert, full JSON response

Privacy / Data Minimization (SABSE IMPORTANT VIVA POINT)

Gemini ko sirf ye bheja jata hai:

```
Monthly Revenue: ₹18000
Monthly Expenses: ₹11000
Profit Margin: 38.89%
Cash Flow Risk: low
Break-Even Target: ₹...
Matched Schemes: Mahila Samridhi Yojana, Stand-Up India
```

Kabhi nahi bheja jata: naam, phone number, email, exact address/village, social category (SC/ST), gender.

System instruction:

"Do NOT change any numbers (amounts in ₹, percentages, units) — keep them exactly as given. Never output PII or fabricate new numbers. Keep the response to 2-3 simple sentences."

temperature: 0.3 — low, creativity kam, consistency zyada.

Graceful degradation: `llmSummaryText` ki initial value hi `financialSummary.explanation` (deterministic template) hai. Agar Gemini fail ho, timeout ho, ya API key hi na ho — `catch` block sirf log karta hai, aur **deterministic English explanation** chala jata hai. **User ko kabhi error nahi dikhta.**

Response shape

```
{
  success: true,
  plan: {
    id, breakEvenUnits, marginPercent, cashFlowRisk, summaryText,
    eligibleSchemes: [{ id, name, benefitSummary, reasons[], applicationLink }],
    allSchemeResults: [...], // ineligible bhi, reasons ke saath
    createdAt
  }
}
```

10.2 POST /api/onboarding/parse AI Extraction

Input: { step, transcript, currentData? }

Kaam: User ne jo bola ("main pandrah hazaar kamata hoon aur das hazaar kharch") usko structured JSON mein badalna:

```
{ "monthly_revenue_est": 15000, "monthly_expense_est": 10000, "confidence": "high" }
```

Gemini configuration

- `model: 'gemini-2.5-flash'` — fast + sasta, extraction ke liye sufficient
- `responseMimeType: 'application/json'` — **structured output mode**, guaranteed valid JSON milta hai
- `temperature: 0.1` — almost deterministic (extraction creative task nahi hai)

System prompt ke 7 critical rules

1. **"NEVER guess, fabricate, or invent values"** — hallucination guard
2. Value na mile to `null`, kuch bhi maan kar mat likho
3. Sector normalization map: silai/kapde → `tailoring`, kirana/dukan → `retail`, doodh/gaay → `dairy`, kheti/kisan → `agriculture`, dhaba/chai → `food`, badhai/handicraft → `manufacturing`, salon/mechanic → `services`
4. Currency: "15 hazaar" / "15k" / "पंद्रह हजार" → `15000`
5. Affirmatives: haan, ha, sahi hai, theek hai, haanji → `true`
6. Negatives: nahi, galat hai → `false`
7. Strict JSON schema with `confidence: high|medium|low`

Fallback Parser — Deterministic Regex (Excellent talking point)

Agar `GEMINI_API_KEY` set hi nahi hai, YA Gemini call fail ho jaye, to `fallbackParser()` chalta hai:

Step	Fallback Logic
name	Regex se prefix hatao: <code>/^(मेरा नाम my name is i am naam नाम)\s*(is hai है :)?\s*/i</code> , phir trailing "hai/है" bhi
district	Similar prefix strip: "main...se", "from", "jilla", "गांव"
sector	7 regex patterns — <code>/tailor silai kapde सिलाई/</code> → tailoring, etc.
finances	<code>/\d+(?:,\d+)*(?:\.\d+)?/g</code> se saare numbers nikalo; pehla = revenue, doosra = expense
loans / confirmation / consent	Token lists se yes/no detect — yesTokens mein 14 variants, noTokens mein 8

Response mein `source: 'gemini' | 'fallback'` return hota hai — debugging ke liye transparent.

Ye batao viva mein: "System LLM ke bina bhi kaam karta hai — degraded quality par, zero downtime." Ye production-thinking dikhata hai.

10.3 POST /api/onboarding/complete DPDP Compliance

Zod schema ka star: consent enforcement

```
consent_given: z.boolean().refine((val) => val === true, {
  message: 'Explicit DPDP Act consent is required before saving your information.',
})
```

Ye legally significant hai. India ka **Digital Personal Data Protection Act, 2023** personal data process karne se pehle *free, specific, informed, unambiguous consent* maangta hai. Humne ise **type system ke andar** enforce kiya hai — `consent_given: false` bhejne par request **validation par hi** 400 se reject ho jati hai, DB tak pahunchti hi nahi. Bypass karna assambhav hai.

Flow

1. Zod validate (consent + name + sector + revenue + expense required)
2. **User resolution:**
 - Agar `user_id` diya hai (logged-in web user) → `users` table mein `upsert` with `onConflict: 'id'`.
Zaroori kyun? Google OAuth se `auth.users` mein row banti hai par `public.users` mein nahi.
`business_profiles.user_id` ka FK `public.users(id)` par hai — wo row pehle honi chahiye warna FK violation.
 - Agar nahi → phone normalize (10 digit → +91 prefix), existing dhundo, warna naya banao. Phone na ho to random +91XXXXXXXXXX temporary generate karte hain.
3. `business_profiles` `upsert (onConflict: 'user_id')` — ek user ka ek profile
4. Financial summary compute + schemes match
5. Initial plan `financial_plans` mein insert
6. Return `{ success, userId, redirectUrl: '/dashboard?user_id=...' }`

10.4 POST /api/business-guide/generate AI Advisory

Input: `{ user_id, challenge_text }` (min 3 chars)

The 5 FIXED Stages (rename/reorder allowed nahi)

#	Stage	Kya karwata hai
1	Cost Optimization लागत में कमी	Group purchasing, direct wholesaler contact, daily waste tracking, 2+ supplier quotes. Target: 10-15% cost reduction.
2	Value Addition मूल्य संवर्धन	Better packaging, grading, bundling, combo offers, loyalty discount. Target: 15-25% higher margin.
3	Direct Market Access सीधी बाज़ार पहुंच	Bichauliye (middlemen) hatao — WhatsApp broadcast catalog, weekly haat stalls, 50 direct families. Target: 25% extra sales.
4	Formal Registration औपचारिक पंजीकरण	Udyam (free MSME registration), FSSAI (₹100 for food), separate business current account. Unlocks PMEGP/Mudra eligibility.
5	Scale विस्तार व बचत	Monthly savings target (max(₹2000, 15% of revenue)), 6 months save karo <i>tab</i> Mudra apply karo, 40% profit reinvest.

Validation of LLM output

```
if (Array.isArray(parsedRoadmap) && parsedRoadmap.length === 5) {
  roadmap = parsedRoadmap.map((item, idx) => ({
    stage: FIXED_STAGES[idx], // ← LLM ka stage name OVERWRITE kar dete hain
    title_hi: item.title_hi || FIXED_STAGES[idx],
    description: item.description || '',
    action_items: Array.isArray(item.action_items) ? item.action_items : [],
    impact_milestone: item.impact_milestone || '',
  }));
} else {
  roadmap = generateFallbackRoadmap(sector, revenue, expense, challenge_text);
}
```

Key defensive pattern: Chahe Gemini ne stage ka naam badal diya ho, hum `FIXED_STAGES[idx]` se overwrite kar dete hain. Har field par `|| default` hai. Aur agar array 5 length ka nahi hai → poora fallback. Matlab **UI ko hamesha exactly 5 valid stages milte hain.**

Fallback roadmap poori Hindi mein hardcoded hai, aur user ke actual numbers use karta hai:
`Math.round(expense * 0.1)` savings target, `max(2000, revenue * 0.15)` monthly savings, `× 6` six-month fund.

10.5 POST /api/ledger/ocr Computer Vision

Input: multipart/form-data — `image` (File) + `user_id` (string)

Flow

- `request.formData()` parse (try/catch)
- File aur `user_id` validate — dono required, warna 400
- `File → arrayBuffer() → Buffer.from()` (Node Buffer)
- `Tesseract.recognize(imageBuffer, 'eng+hin', { logger: () => {} })` — dono language packs, progress logs suppressed
- `parseOcrText(rawText)` — 3 regex patterns (neechे)

6. `ledger_entries` mein bulk insert with `confirmed: false, source: 'ocr'`

7. Return `rawText` + `parsedEntries` + `savedCount`

The 3 OCR Regex Patterns

Pattern	Regex	Matches
1. Word + Number	<code>/^(.+?)\s+([0-9,]+(?:\.[0-9]{1,2})?)\s*(?:\s \/ \$)/</code>	"Rice 150", "Labour charges 500.00". <code>.+?</code> non-greedy hai taaki last number capture ho.
2. Label: Number	<code>/^([A-Za-z\s]+):\s*(?:Rs\.[0-9]{1,2} ₹)?\s*([0-9,]+\.[0-9]{1,2})/i</code>	"Total: 1500", "Amount: ₹250.00"
3. Standalone	<code>/^(?:Rs\.[0-9]{1,2} ₹)?\s*([0-9,]+(?:\.[0-9]{1,2})?)\$/</code>	"₹1,500" akela line par

Sanity filters

- `amount > 0 && amount < 10000000` — ₹1 crore se upar micro-business mein galat OCR hai
- `!/^\d{1,2}[-\/]\d{1,2}/.test(description)` — "12/05 500" jaisi date lines ko skip karta hai
- Commas strip: `replace(/,/g, '')` before `parseFloat`

Limitations (honestly bolo): (a) Saari entries default `entry_type: 'income'` par set hoti hain — OCR income vs expense distinguish nahi kar sakta, isliye user ko dashboard par correct karna padta hai. (b) Handwritten Devanagari par Tesseract ki accuracy kam hai (printed text par acchi hai). (c) Insert fail hone par bhi non-fatal hai — parsed data return ho jata hai. **Isliye `confirmed: false` design hai — human-in-the-loop.**

10.6 /api/whatsapp/webhook Integration

GET — Meta Verification Handshake

```
const mode = searchParams.get('hub.mode'); // 'subscribe'
const token = searchParams.get('hub.verify_token'); // humara secret
const challenge = searchParams.get('hub.challenge'); // random string

if (mode === 'subscribe' && token === process.env.WHATSAPP_VERIFY_TOKEN) {
  return new NextResponse(challenge, { status: 200,
    headers: { 'Content-Type': 'text/plain' } }); // PLAIN TEXT, JSON nahi!
}
return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
```

Kyun: Meta proof maangta hai ki ye endpoint hamara hai. Wo random challenge bhejta hai; hum verify token match karke challenge **as-is plain text** wapas karte hain. JSON return karna galat hai — verification fail ho jayega.

POST — Message Processing

```
processWhatsAppMessages(body).catch(err => console.error(err)); // NO await
return NextResponse.json({ status: 'received' }, { status: 200 }); // instant
```

Fire-and-forget: Meta 20s timeout ke andar 200 chahta hai, warna wo message retry karega → duplicate processing. Isliye response turant, processing background mein.

Nested payload traversal: `body.entry[] → changes[] → value.messages[]` . Har level par optional chaining + default arrays — Meta ka payload shape vary karta hai.

Media handling: Image aaye to pehle `getWhatsAppMediaUrl(mediaId)` se Graph API se actual download URL resolve karna padta hai (WhatsApp direct URL nahi bhejta).

Reply: `POST graph.facebook.com/v19.0/{phoneNumberId}/messages` with Bearer token. `+` prefix hata kar bhejte hain (WhatsApp `919876543210` format chahta hai).

10.7 POST /api/sms/webhook Integration

Twilio `application/x-www-form-urlencoded` bhejta hai. Fields: `From` , `Body` .

Double fallback parsing

```
try { formData = await request.formData(); }
catch {
  const rawText = await request.text();
  const params = new URLSearchParams(rawText); // manual parse
}
```

Twiml Response (XML)

```
<?xml version="1.0" encoding="UTF-8"?>
<Response><Message>{escaped text}</Message></Response>
```

XML Injection prevention: `twimlResponse()` 5 characters escape karta hai: `& → &` , `< → <` , `> → >` , `" → "` , `' → '` . **Order important hai:** `&` sabse pehle escape hota hai, warna `<` ;dobara escape hokar `&lt;` ; ban jata. Agar attacker apna naam `</Message><Message>spam` rakh de to wo Twilio ko extra SMS bhejne par majboor kar sakta tha — ye escaping usse rokti hai.

WhatsApp vs SMS difference: WhatsApp asynchronous hai (200 OK phir separate API call se reply), SMS synchronous hai (reply usi HTTP response body mein Twiml ke roop mein).

10.8 GET /api/keepalive Ops

Supabase free tier inactivity ke baad project pause kar deta hai. UptimeRobot har 5 min ye endpoint hit karta hai.

```
const { error } = await supabaseServer.from('schemes')
  .select('id', { count: 'exact', head: true }); // head:true = no rows, only count
return NextResponse.json(
  { status: 'ok'|'degraded', timestamp, scheme_count, db },
  { headers: { 'Cache-Control': 'no-store, no-cache, must-revalidate' } }
);
```

`head: true` — sirf count chahiye, actual rows nahi (bandwidth bachata hai). `no-store` header taaki CDN cache na kare — warna DB hit hi nahi hoga aur keepalive ka matlab hi khatm.

Note: Ye endpoint DB down hone par bhi **HTTP 200** return karta hai (body mein `status: 'degraded'`). Design choice — UptimeRobot ko "site down" alert na mile jab sirf DB slow ho.

10.9 POST /api/facilitator/add-entrepreneur

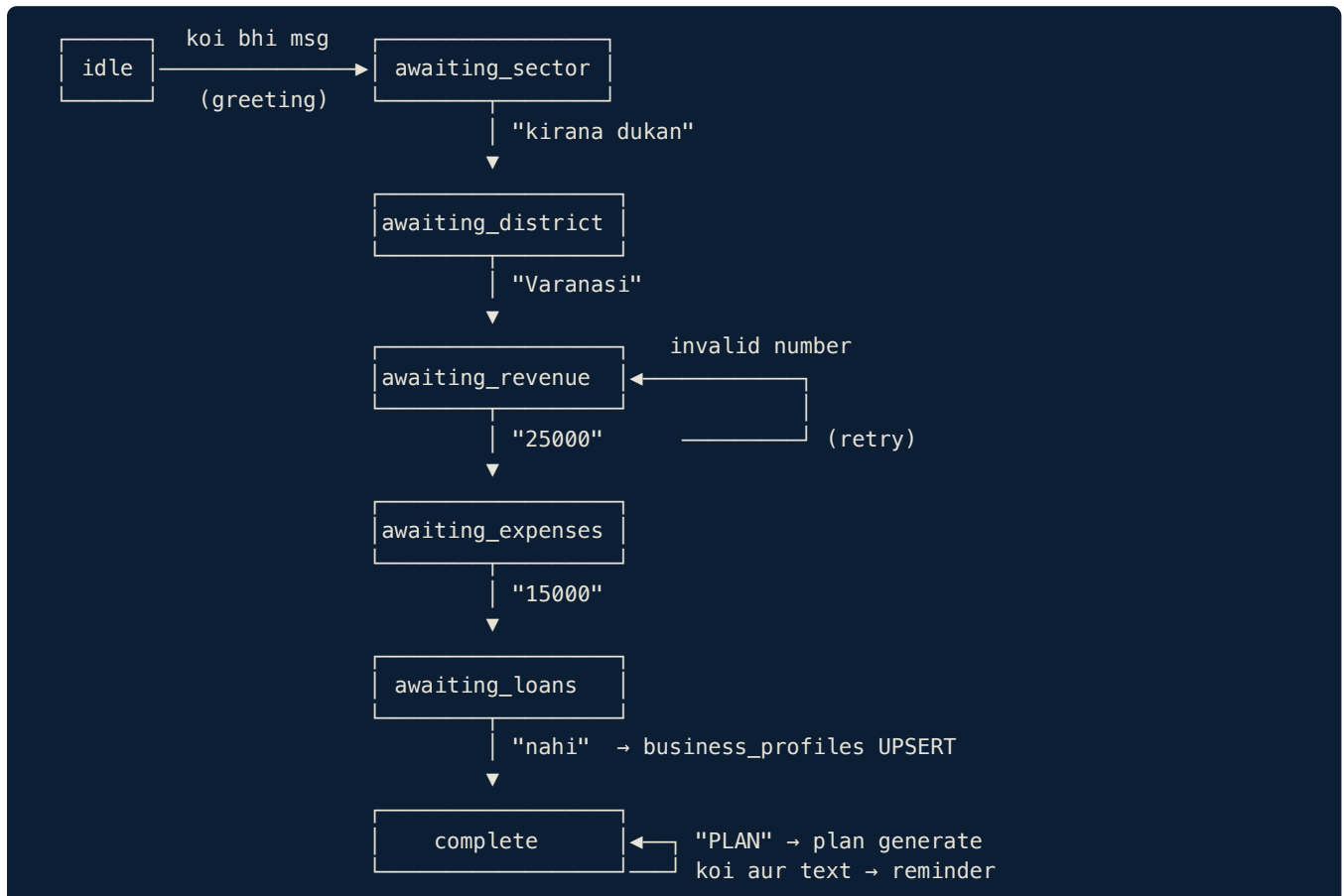
Field worker (NGO/bank mitra) offline mile entrepreneur ko system mein add karta hai.

1. Zod: phone (min 10), name?, sector?, facilitator_id?
2. Phone normalize: `replace(/\D/g, '')` → 10 digit ho to `+91` prefix
3. User find-or-create
4. Sector diya ho to `business_profiles` upsert
5. `facilitator_id` diya ho to `facilitators_entrepreneurs` upsert with `onConflict: 'facilitator_id,entrepreneur_id'` (composite unique key)

11. Conversation Orchestrator — State Machine

src/lib/orchestrator/conversationOrchestrator.ts — 378 lines. WhatsApp aur SMS dono ka shared brain.

11.1 The 6-State FSM



11.2 Execution Steps (har message par)

1. **Upsert user** — phone se `users` dhundo. Error code `PGRST116` = "no rows found" (ye error nahi hai, expected hai). Na mile to create karo with `language: 'hi', role: 'entrepreneur'`.
2. **Fetch/create conversation** — `user_id + channel` se latest conversation. Na ho to `state: 'idle', context: {}` se banao.
3. **Media check** — agar `mediaUrl` hai, kisi bhi state mein, to "bill padh rahe hain" reply. (OCR separately handle hota hai.)
4. **Switch on state** — respective handler chalo, context update karo, next state save karo, reply return karo.

11.3 Helper Functions

parseAmount()

```
const cleaned = text.replace(/[₹,\s]/g, '').trim();
const num = parseFloat(cleaned);
return isNaN(num) || num < 0 ? null : num;
```

"₹ 15,000" → "15000" → 15000. Negative aur NaN dono reject.

parseYesNo()

```
yesVariants = ['yes', 'haan', 'haa', 'ha', 'हाँ', 'हां', 'ह़ा', 'y', '1']
noVariants = ['no', 'nahi', 'nai', 'नहीं', 'नही', 'n', '0']
if (yesVariants.some(v => lower === v || lower.startsWith(v))) return true;
```

Devanagari + Roman + Hinglish sab. `startsWith` se "haan ji bilkul" bhi match karta hai.

11.4 Why Persist State in DB? (Great architecture question)

Vercel par har API call ek **stateless serverless function invocation** hai. Do messages ke beech koi shared memory nahi hoti — process hi alag ho sakta hai.

Isliye conversation ka poora state (`state` + `context` JSONB) har turn par **DB mein persist** hota hai. Ye system ko:

- **Horizontally scalable** banata hai — koi bhi server instance koi bhi message handle kar sakta hai
- **Crash-resilient** — server restart hone par bhi user ka progress nahi jata
- **Auditable** — poori conversation history DB mein hai

11.5 Bilingual Message Dictionary

`t(lang, key)` function — nested lookup with **double fallback**:

```
return messages[lang]?.[key] ?? messages['hi'][key] ?? key;
```

Unknown language → Hindi. Unknown key → key name khud (developer ko turant dikh jayega ki translation missing hai).

Keys: `ask_sector`, `ask_district`, `ask_revenue`, `ask_expenses`, `ask_loans`, `complete`, `plan_generating`, `plan_error`, `invalid_number`, `invalid_yesno`, `ocr_processing`, `ocr_error` — Hindi aur English dono mein.

11.6 Known Limitation (honestly batao)

complete state mein jab user "PLAN" bhejta hai, `callPlanGenerate()` async fire hota hai aur user ko turant "⌚ plan ban raha hai" reply milta hai. Par jab plan **actually ban jata hai**, to use wapas bhejne ka mechanism nahi hai — code mein comment bhi likha hai: *"Can't easily send a delayed message back in this architecture."*

Plan DB mein save ho jata hai (user web par dekh sakta hai), par WhatsApp par push nahi hota.

Fix kaise hoga (ye bolna impressive lagega): Ek job queue (BullMQ/Inngest/QStash) chahiye jo plan generate hone par ek **outbound WhatsApp send** trigger kare. Ya phir WhatsApp Business Template Message use karein session window ke bahar bhejne ke liye.

12. Voice AI — Web Speech API

src/components/VoiceOnboardingModal.tsx — 832 lines

12.1 Do alag browser APIs

API	Direction	Config in our code
SpeechRecognition / webkitSpeechRecognition	Speech → Text (STT)	continuous = false (ek sentence sun kar ruk jao), interimResults = true (live transcript dikhao jaise-jaise bole), lang = 'hi-IN' (Hindi + Hinglish dono handle karta hai)
speechSynthesis + SpeechSynthesisUtterance	Text → Speech (TTS)	utterance.lang = 'hi-IN' , available voices mein se hi-IN ya en-IN wali dhundhta hai

Browser detection: 'webkitSpeechRecognition' in window || 'SpeechRecognition' in window .
Chrome/Edge support karte hain; **Firefox nahi karta**, iOS Safari partial. Isliye text form fallback mandatory hai.

12.2 The 9-Step Flow

Step	Hindi Prompt (bola jata hai)	Validation
auth	"कृपया अपना 10 अंकों का मोबाइल नंबर दर्ज करें"	replace(/\D/g, '').length >= 10 , last 10 digits lete hain
name	"नमस्ते! आपका शुभ नाम क्या है?"	Gemini parsed.name ya raw transcript
district	"आप किस गांव या जिले में रहते हैं?"	parsed.district parsed.village
sector	"आपका क्या काम या व्यापार है?"	parsed.sector — normalized identifier
finances	"हर महीने कितनी कमाई और कितना खर्च?"	rev > 0 hona chahiye, warna retry
loans	"क्या पहले से कोई कर्ज है?"	parsed.existing_loans ya regex /yes haan हाँ/i
confirmation	"क्या सभी जानकारी सही है?"	No bola to wapas 'name' step par — correction allow
consent	"क्या आप यह जानकारी सेव करने की अनुमति देते हैं?"	No → "DPDP सहमति के बिना जानकारी सेव नहीं की जा सकती"
complete	"बधाई हो! डैशबोर्ड पर ले जा रहे हैं..."	2s delay phir router.push

12.3 Layered Robustness (3 levels)

Level 1: Gemini extraction (best)	→ parsed.name, parsed.sector...
Level 2: Regex fallback parser	→ server-side, if Gemini fails
Level 3: Raw transcript as-is	→ `parsed.name    rawInput.trim()`
Level 4: Manual text input box always visible	→ mic fail ho to type kar do

Har step par ye chain chalti hai. Iska matlab — **voice onboarding kabhi completely fail nahi hota.**

12.4 React Patterns Used

- `useRef` for recognition instance — re-render par naya object na bane
- `useCallback` for `speakText`, `startListening`, `stopListening`, `changeStep` — stable references, `useEffect` dependency loops se bachne ke liye
- `window.speechSynthesis.cancel()` cleanup mein — modal band karne par bolna ruk jaye
- Functional state updates: `setFormData(prev => ({...prev, name}))` — stale closure bug se bachav

13. Authentication Flow

13.1 Three Supabase Clients — Kab Kaunsa

Client	Key	RLS	Kab use karein
<code>supabaseClient</code> <code>client.ts</code>	ANON	✅ ON	Client Components ('use client'). Browser mein chalta hai. <code>createBrowserClient</code> from <code>@supabase/ssr</code> — session cookies mein.
<code>supabaseServer</code> <code>server.ts</code>	SERVICE_ROLE	❌ BYPASS	API routes, Server Components mein jab admin access chahiye (webhooks, cross-user queries). Kabhi browser par nahi.
<code>createSupabaseRouteClient()</code> <code>server.ts</code>	ANON + cookies	✅ ON	Server-side par jab jaanna ho " kaun logged in hai ". Request ke cookies se bound hota hai.

Cookies vs localStorage — critical decision

`client.ts` mein plain `@supabase/supabase-js` ke bajaye `createBrowserClient` from `@supabase/ssr` use kiya, kyunki:

- **localStorage** sirf browser mein hota hai — server usse nahi padh sakta. To Server Components ko pata hi nahi chalta ki user logged in hai.
- **Cookies** har request ke saath automatically server tak jate hain. Isse `/dashboard` (Server Component) aur `/auth/callback` (Route Handler) dono user ko pehchan sakte hain.
- Aur `localStorage` hone par Supabase default **implicit flow** use karta, PKCE nahi — jisse `/auth/callback` ka code exchange hi toot jata.

13.2 proxy.ts — Session Refresh Middleware

```
export async function proxy(request: NextRequest) {
  let response = NextResponse.next({ request });
  // ... createServerClient with cookie read/write handlers ...
  try {
    await supabase.auth.getUser(); // ← ye call token refresh trigger karta hai
  } catch (err) {
    console.warn('Proxy session refresh failed, continuing unauthenticated', err);
  }
  return response;
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp|ico)$).*'],
};
```

3 important points

1. **Next.js 16** mein "**middleware**" ka naam "**proxy**" ho gaya — file `src/proxy.ts`, exported function `proxy()`. Pehle ye `middleware.ts` / `middleware()` tha.

2. **getUser()** ka side effect: Agar access token expire ho gaya par refresh token valid hai, to ye silently naya token le aata hai aur **setAll** handler ke through cookie rewrite kar deta hai. Iske bina user ko random logouts milte.
3. **try/catch kyun**: Ye *har* matched request par chalta hai. Agar Supabase Auth service 2 second ke liye down ho, aur ye throw kare, to **poori website 500 de deti**. Isliye catch karke unauthenticated continue karte hain.

Matcher regex negative lookahead use karta hai — static files, images, favicon skip karta hai (performance).

13.3 OAuth PKCE Flow (Google Sign-In)

```
1. User "Sign in with Google" click karta hai
2. supabaseClient.auth.signInWithOAuth({ provider: 'google',
  options: { redirectTo: `${origin}/auth/callback` } })
  → PKCE: code_verifier generate hota hai, uska SHA256 hash (code_challenge) bhejte hain
3. Google par login → Google redirect karta hai /auth/callback?code=xxx
4. Route handler: supabase.auth.exchangeCodeForSession(code)
  → code + stored code_verifier bhejta hai, session cookies milti hain
5. Smart routing:
  business_profiles mein row hai? → /dashboard
  nahi hai? → /onboarding
6. NextResponse.redirect with pendingCookies attached
```

PKCE (Proof Key for Code Exchange) kyun? Public clients (browser apps) mein client_secret safely store nahi ho sakta. PKCE mein har login ke liye random **code_verifier** banta hai. Agar attacker authorization code chura bhi le, wo bina verifier ke session exchange nahi kar sakta. RFC 7636.

Callback route ke 4 defensive layers

1. OAuth error params (**error**, **error_description**) aaye to login par friendly message ke saath redirect
2. Placeholder/demo env (Supabase configure nahi) → seedha next par redirect (demo chalta rahe)
3. Real code → exchange. Profile lookup **try/catch mein wrapped** — comment mein likha hai: *"This check must never block the redirect itself"*. Lookup slow/fail ho to default destination par bhej do, sign-in stall na ho.
4. Kuch na mile → **/login?error=auth_failed**

Production host handling: **x-forwarded-host** header padhkar redirect origin banate hain — Vercel ke reverse proxy ke peeche **origin** internal URL hota hai, public domain nahi.

13.4 /auth/confirm — Email Confirmation (3 fallback flows)

1. **Flow 1:** **token_hash** + **type** → **verifyOtp({ type, token_hash })** — standard Supabase email link
2. **Flow 2:** **code** → **exchangeCodeForSession(code)** — PKCE email link
3. **Flow 3:** **token** + **type** + **email** → **verifyOtp({ type, token, email })** — legacy format

Teeno try hote hain sequence mein; koi bhi succeed kare to **/login?confirmed=true**. Sab fail → **/login?error=confirmation_failed**. **Kyun 3?** Supabase ne format badle hain versions ke saath, aur purane bheje gaye emails bhi kaam karne chahiye.

14. Frontend Pages

14.1 Server Components vs Client Components

Page	Type	Kyun
/dashboard	Server	DB se directly fetch karta hai — koi loading spinner nahi, koi client-side API call nahi. Service-role key safely use kar sakta hai. <code>export const dynamic = 'force-dynamic'</code> — har request par fresh data (static cache nahi).
/facilitator	Server	Same — saare entrepreneurs ka data server par aggregate hota hai.
/dashboard/schemes	Client	Interactive: eligible/ineligible toggle, document checkboxes, "Recheck" button with <code>useTransition</code> .
/dashboard/business-guide	Client	Voice input, form submit, live generation state.
/onboarding, /login	Client	Forms, speech API, router navigation.
/ (landing)	Client	Persona filter buttons, FAB, modal state.

14.2 Dashboard — 4-Level User Resolution Fallback

1. Authenticated session cookie se user (`createSupabaseRouteClient → auth.getUser()`)
↓ agar `public.users` mein row nahi hai, upsert kar do
 2. `?user_id=` query param se (onboarding ke baad redirect isi ke saath aata hai)
 3. DB ka latest created user (demo/testing convenience)
 4. Hardcoded demo user "Ramesh Kumar" — DASHBOARD KABHI BLANK NAHI HOTA

Ye pattern demo-resilience ke liye hai. SIH judging mein agar network/DB hiccup ho, dashboard phir bhi meaningful UI dikhata hai. *Par honestly bolo ki production mein level 3 aur 4 hata dene chahiye* — "latest user" dikhana privacy issue hai. MVP/demo trade-off hai.

Plan: stored ya live-computed

Agar `financial_plans` mein row hai → wahi dikhao. Nahi hai → `calculateMarginPercent()` + `assessCashFlowRisk()` + `matchSchemes()` **on-the-fly** chala kar dikha do. Schemes bhi DB se na mile to `FALLBACK_SCHEMES` array (5 hardcoded schemes: PMEGP, Mudra Shishu, PM SVANidhi, Stand-Up India, Mudra Kishor) use hota hai.

15-day ledger chart

```
const days = Array.from({ length: 15 }, (_, i) => { ... });
```

`ledger_entries` se pichle 15 din ka income/expense aggregate karke bar chart banta hai — pure CSS/SVG, koi chart library nahi (bundle size chhota).

14.3 Yojana Kendra (/dashboard/schemes)

- `getSchemeDetails(schemeName)` — scheme name par `includes()` matching karke sponsoring ministry + required documents list return karta hai. E.g. PMEGP → "KVIC / Ministry of MSME" + 6 documents (Aadhaar, Project Report, Caste Certificate, 8th marksheet, Bank passbook, Photos). Har list **Hindi + English dono** mein.
- **Document checklist** — `toggleDocCheck(docId)` se user tick kar sakta hai kaunsa document ready hai
- **Recheck button** — `useTransition()` ke andar schemes dobara fetch + rematch karta hai (profile update hone ke baad)
- **Show ineligible toggle** — default chhupe rehte hain, par expand karke reasons dekh sakte hain

14.4 Internationalization (i18n)

- `lib/i18n.ts` — `translations: Record<'en'|'hi', Record<string, string>>`, ~165 keys per language
- `contexts/LanguageContext.tsx` — React Context, `useState<Language>('en')`, `toggleLanguage()`, aur `t(key)` function
- `t()` ka fallback: `langMap[key] ?? key` — missing translation par key return, crash nahi
- `LanguageToggleButton` — floating pill 🌐 button, har page par

Limitation: Language state `localStorage` /cookie mein persist nahi hoti — page refresh par wapas 'en' ho jati hai. Easy fix, par abhi nahi kiya.

14.5 Design System

Element	Value
Background	<code>#F5F1E6</code> warm cream — sunlight mein readable (rural outdoor use!)
Text / Ink	<code>#0B1E33</code> deep navy — high contrast (WCAG AA+)
Accent	<code>#C9A24B</code> gold
Display font	Playfair Display (serif) — headlines
Body font	Inter / Plus Jakarta Sans — readability
Custom fonts	Bodoni Moda, EB Garamond, Space Grotesk, Material Symbols — self-hosted <code>.ttf</code> in <code>/public/fonts</code> with <code>font-display: swap</code>
Corners	32-40px rounded on cards/images

Design rationale: High-contrast + large text kyunki target users bright sunlight mein sasta phone use karte hain, aur kai log low-literacy hain.

15. Testing

15.1 Kya test hua

File	Tests	Coverage
financialEngine.test.ts	~28	Saare 4 functions, boundary values, edge cases
schemeMatcher.test.ts	8	Real profile scenarios, sorting, reason strings

Command: `npx vitest run` | **Watch:** `npx vitest` | **UI:** `npx vitest --ui`

15.2 Kya test NAHI hua (honestly bolo)

- **API routes** — koi integration test nahi. Supabase ko mock karna padta (MSW/nock ke saath).
- **React components** — koi component test nahi. `@testing-library/react` chahiye hota.
- **E2E** — Playwright/Cypress nahi hai.
- **Conversation orchestrator** — DB dependency ki wajah se untested (isliye pure logic `parseAmount` / `parseYesNo` ko alag file mein extract karke test kiya ja sakta tha).

Ye defend kaise karein: "Humne testing effort wahan concentrate kiya jahan **galti sabse mehngi** hai — financial calculations aur eligibility decisions. UI bug se user ko dikkat hoti hai; galat break-even number se user galat business decision le sakta hai. Isliye engines 100% tested hain aur pure functions ke roop mein design kiye gaye taaki testable rahein. API routes ke liye next step MSW-based integration tests hain."

16. Deployment & DevOps

16.1 Setup Sequence

- 1. `npm install`
- 2. Supabase project banao → SQL Editor mein **order se** chalao:
`001_init.sql` → `seed/schemes.sql` → `002_rls_policies.sql` → `004_business_guides.sql`
(ya `supabase link --project-ref XXX` phir `supabase db push`)
- 3. `.env.local` banao — 11 environment variables
- 4. `npx vitest run` — tests pass hone chahiye
- 5. `npm run dev` → localhost:3000
- 6. GitHub par push → Vercel import → env vars add → Deploy
- 7. Meta webhook configure: `https://domain/api/whatsapp/webhook` , verify token, subscribe to `messages` field
- 8. Twilio: phone number → Messaging → "A message comes in" → `https://domain/api/sms/webhook` , POST
- 9. UptimeRobot: HTTP(s) monitor on `/api/keepalive` , 5 min interval

16.2 Scripts

Command	Kaam
<code>npm run dev</code>	Development server (Turbopack)
<code>npm run build</code>	Production build
<code>npm start</code>	Production server
<code>npm run lint</code>	ESLint
<code>npx vitest run</code>	Unit tests

17. Limitations & Future Scope

Examiner **zaroor** puchega "kya limitations hain?" — jo team apni limitations jaanti hai wo mature lagti hai. Ye list yaad rakho.

17.1 Known Limitations

Limitation	Detail	Mitigation / Future fix
Break-even approximation	Per-unit price/cost data onboarding par nahi milta, isliye true unit break-even nahi nikalta	Optional "advanced" onboarding step jisme per-unit data poocha jaye
WhatsApp delayed reply	Plan generate hone ke baad user ko WhatsApp par push nahi hota	Job queue (Inngest/QStash) + WhatsApp template message
Service-role key bypasses RLS	API routes mein security application layer par depend karti hai	Per-request JWT-bound clients jahan possible ho; RLS ko last line of defense rakho
OCR entry_type always 'income'	Income/expense distinguish nahi kar sakta	Already mitigated by <code>confirmed: false</code> — user correct karta hai. Future: keyword classifier
OCR handwriting accuracy	Tesseract printed text par accha, handwritten Devanagari par weak	Cloud Vision API ya fine-tuned model (par privacy trade-off)
Dashboard demo fallbacks	Level 3/4 fallback "latest user" dikha sakta hai	Production mein remove + proper auth redirect
Language not persisted	Refresh par EN par reset	localStorage / cookie / user preference in DB
State name matching	"UP" vs "Uttar Pradesh" match nahi karega	Normalization dictionary
No API tests	Sirf engines tested hain	MSW + Vitest integration tests
Web Speech browser support	Firefox support nahi karta	Already mitigated — text form fallback
Schemes manually seeded	Government scheme rules badalte hain, humara data stale ho sakta hai	Admin panel + periodic review workflow; ya government API integration
groq-sdk unused	Dependency installed hai par use nahi hoti	Remove kar dena chahiye (bundle hygiene)

17.2 Future Scope (roadmap)

- IVR / voice call channel** — jinke paas WhatsApp bhi nahi (Twilio Voice / Exotel)
- More Indian languages** — LANGUAGE_NAMES map mein already 11 hain (ta, te, mr, bn, gu, kn, ml, pa, or); UI translations add karna baaki
- Credit score proxy** — ledger consistency se alternative creditworthiness signal (banks ko dikhane ke liye)
- Bank/NBFC partnership API** — direct application submission
- Offline-first PWA** — poor connectivity areas ke liye

6. **Peer benchmarking** — "aapke jaise kirana stores ka average margin 22% hai" (anonymized aggregate)
7. **Automated scheme sync** — government portals se rules scrape/API
8. **Facilitator mobile app** — offline data collection

18. VIVA QUESTION BANK — 130+ Q&A

Ye section sabse important hai. Har answer **bolne layak** likha hai — ratta nahi, samajh kar bolo.

A. Project Overview (Q1-Q12)

Q1. Apna project 60 seconds mein explain karo.

Saathi Vyapar ek AI-powered financial advisory platform hai rural micro-entrepreneurs ke liye. Problem ye hai ki India ke 6 crore+ micro-businesses ke paas na bookkeeping hai, na koi advisor, aur na government schemes ki visibility. Humara solution teen channels par kaam karta hai — WhatsApp, SMS aur web. Core mein do deterministic engines hain: ek financial engine jo break-even, profit margin aur cash-flow risk calculate karta hai pure math se, aur ek scheme matcher jo 15 real government schemes ke against eligibility check karta hai aur har decision ka human-readable reason deta hai. AI ka use hum sirf language ke liye karte hain — numbers kabhi LLM se nahi aate. Tech stack Next.js 16, React 19, TypeScript, Supabase Postgres with Row Level Security, aur Google Gemini 2.5 Flash hai.

Q2. "Saathi Vyapar" naam ka kya matlab hai?

Saathi = companion, Vyapar = business. "Business Companion". Deliberately Hindi naam rakha kyunki target user rural Hindi-speaking entrepreneur hai — naam khud accessibility ka signal deta hai.

Q3. Ye SIH ka kaunsa problem statement hai?

SIH26091, Ministry of Social Justice & Empowerment. Isliye humne specifically us ministry ki schemes include ki hain — PM-AJAY, NSFDC, NSTFDC, Mahila Samridhi Yojana.

Q4. Aapka target user exactly kaun hai?

Rural micro-entrepreneur — kirana shop owner, tailor, dairy farmer, artisan, street vendor. Monthly revenue typically ₹10,000 se ₹50,000. Aksar low-literacy, feature phone ya sasta smartphone, intermittent internet. Secondary user: field facilitator (NGO worker, bank mitra, SHG coordinator) jo unhe onboard karta hai.

Q5. Existing solutions se aap kaise alag ho?

Teen tarah se. Ek — Jan Samarth/UMANG sirf loan discovery karte hain, hum full financial advisory dete hain (break-even, margin, risk). Do — wo sab smartphone + app + reading ability maangte hain; hum SMS aur voice par chalte hain. Teen — wo binary eligible/not-eligible batate hain; hum har decision ka reason batate hain, jisse user gap close kar sake.

Q6. Ye "AI project" hai ya nahi? Kitna AI hai?

Ye deliberately hybrid hai. AI three specific tasks karta hai — natural language extraction (bola hua vaakya se structured data), translation/rephrasing (numbers ko vernacular mein samjhana), aur qualitative advisory (growth roadmap). Par saare **quantitative decisions** deterministic code se hote hain. Ye conscious architecture choice hai — financial domain mein hallucination acceptable nahi hai.

Q7. Multi-channel kyun? Sirf app kyun nahi?

Kyunki target user ke paas app install karne ki capability ya smartphone hi nahi ho sakta. SMS ka matlab hai bilkul feature phone par bhi, bina internet ke service milegi. WhatsApp India mein 50 crore+ users ke paas hai — already installed, already familiar. Web unke liye jo comfortable hain ya facilitator ke liye.

Q8. Scalability kaise handle karoge?

Architecture already stateless hai — saara conversation state DB mein persist hota hai, memory mein nahi. Vercel serverless functions horizontally auto-scale karti hain. Supabase Postgres connection pooling deta hai. Bottleneck Gemini API rate limits honge — uske liye request queuing aur caching (same profile ka summary reuse) add karenge.

Q9. Business/revenue model kya hai?

Ye public-benefit project hai (government problem statement). Sustainability models: (a) government/CSR funding as digital public infrastructure, (b) bank/NBFC partnership — verified lead generation, (c) NGO/SHG federations ke liye facilitator SaaS licensing. End user ke liye hamesha free rehna chahiye.

Q10. Kitna code likha hai?

~10,300 lines TypeScript/TSX across 43 source files. Isme ~430 lines test code hai aur ~333 lines core engines. Plus SQL migrations aur 15-scheme seed data.

Q11. Team mein kisne kya kiya? (apne hisaab se customize karo)

Ready rakho: backend/engines, database+security, frontend/UI, integrations (WhatsApp/Twilio/OCR), AI prompting. Specific bolo, "sab ne sab kiya" mat bolo.

Q12. Sabse bada technical challenge kya tha?

LLM ko financial numbers se door rakhte hue bhi usse useful banana. Pehle socha tha pura plan LLM generate karega, par testing mein wo numbers badal deta tha — kabhi round kar deta, kabhi galat percentage nikaal deta. Isliye architecture flip kiya: pehle deterministic engine se exact numbers calculate karo, phir LLM ko sirf "in numbers ko warm Hindi mein 2-3 line mein likho, numbers mat badlo" ka kaam do. Aur fallback rakha ki LLM fail ho to deterministic explanation chala jaye.

B. Financial Engine (Q13-Q30)

Q13. Break-even formula likho aur derive karo.

Break-even par Total Revenue = Total Cost. Yani $P \times Q = FC + VC \times Q$. Rearrange: $Q(P - VC) = FC$, to $Q = FC / (P - VC)$. Denominator $(P - VC)$ contribution margin hai.

Q14. Contribution margin ka business matlab kya hai?

Ek unit bech kar variable cost nikalne ke baad jo bachta hai — wo paisa fixed costs cover karne mein "contribute" karta hai. Jab tak total contribution fixed costs se kam hai, business loss mein hai.

Q15. Agar price = variable cost ho to kya hoga?

Contribution margin zero. Code `Infinity` return karta hai — matlab kitna bhi becho, fixed cost kabhi cover nahi hogi. Explicit guard hai: `if (contributionMargin <= 0) return Infinity`.

Q16. Negative contribution margin par bhi Infinity kyun, negative number kyun nahi?

Negative break-even ka koi business meaning nahi — "minus 50 units becho" absurd hai. Infinity sahi semantic hai: "impossible". UI mein `isFinite()` check se usko "cannot be calculated" message banate hain.

Q17. Profit margin formula?

$(\text{Revenue} - \text{Expenses}) / \text{Revenue} \times 100$. Denominator revenue hai, cost nahi — ye *net profit margin* hai, markup nahi.

Q18. Margin aur markup mein kya farak hai?

Margin = profit / **revenue**. Markup = profit / **cost**. ₹100 ka saman ₹150 mein becha: margin 33.3%, markup 50%. Hum margin use karte hain — standard financial reporting metric.

Q19. Revenue zero ho to margin function 0 return karta hai — ye galat nahi?

Mathematically $-\infty$ hota, par wo user ko dikhana bekaar hai. Isliye 0 return karte hain *aur* saath mein `assessCashFlowRisk()` `revenue=0` par turant 'high' risk deta hai. Dono milkar sahi signal dete hain: 0% margin + HIGH risk.

Q20. Cash flow risk ke 3 levels aur criteria batao.

HIGH: revenue zero, ya expense ratio ≥ 0.9 , ya (loan hai AND ratio ≥ 0.7). MEDIUM: ratio ≥ 0.7 , ya koi bhi loan. LOW: baaki sab. Order matter karta hai — checks top-down evaluate hote hain.

Q21. 0.7 aur 0.9 thresholds kahan se aaye? Justify karo.

Microfinance practice se liye heuristics hain — 30%+ surplus healthy working-capital buffer, 10% se kam surplus distress zone. Ye hardcoded magic numbers nahi balki tunable policy constants hain; sector-wise calibration future work hai. Conservative aur explainable rakha hai taaki user ko reason samajh aaye.

Q22. Loan hone par threshold 0.9 se 0.7 kyun gir jata hai?

Compound risk. Loan matlab ek **fixed, non-negotiable** monthly obligation — EMI. Agar pehle se 70% income kharch ho rahi hai aur upar se EMI hai, to ek bura mahina default mein badal sakta hai.

Q23. Loan hone par kabhi 'low' risk aa sakta hai?

Nahi. Rule 4 ensure karta hai ki `existingLoans === true` ho to minimum 'medium' milega, chahe ratio kitna bhi accha ho. Debt hamesha risk add karta hai.

Q24. generateFinancialSummary mein variableCost=0 kyun pass kiya?

Onboarding par per-unit data collect nahi karte — user sirf total monthly revenue aur expense batata hai. To true unit break-even calculate nahi ho sakta. Approximation kiya: `fixedCosts = monthly expenses`, price proxy = revenue, `variableCost = 0`. **Isliye output ko "units" ki tarah dikhaya hi nahi** — explanation mein revenue target batate hain: "₹11,000 monthly revenue maintain karna hoga costs cover karne ke liye." Ye code ke comment mein documented hai; future improvement onboarding mein per-unit question add karna hai.

Q25. Financial engine mein koi import kyun nahi hai?

Deliberate. Zero dependencies matlab pure functions, no side effects, same input hamesha same output. 100% unit testable — koi mocking nahi, koi DB setup nahi. Aur code kahin bhi reuse ho sakta hai.

Q26. Pure function kya hoti hai?

Do properties: (1) same input par hamesha same output, (2) koi side effect nahi — DB write nahi, network call nahi, global mutation nahi, `Date.now()` / `Math.random()` nahi. Humare dono engines pure hain.

Q27. toFixed(2) aur parseFloat dono kyun use kiye?

`toFixed(2)` string return karta hai ("38.89"). Humein number chahiye JSON response aur NUMERIC DB column ke liye, isliye `parseFloat()` se wapas number banate hain. Net effect: 2 decimal par rounded number.

Q28. Boundary value testing kya hai, aapne kaise kiya?

Bugs hamesha boundaries par hote hain — `>` vs `>=` ki galti. Humne exactly 0.69 (low expect), exactly 0.70 (medium expect), aur exactly 0.90 (high expect) test kiye hain. Koi galti se `>` likh deta to ye tests turant pakad lete.

Q29. toBeCloseTo kyun use kiya kuch tests mein?

Floating point mein `0.1+0.2 !== 0.3` hota hai. Jahan fractional result aata hai ($10000/1500 = 6.666\dots$), exact equality fail ho sakti hai. `toBeCloseTo(6.667, 2)` 2 decimal tolerance ke saath compare karta hai.

Q30. Naya metric add karna ho, jaise "runway months", kaise karoge?

Nayi pure function `calculateRunway(savings, monthlyBurn)` financialEngine mein add karunga with divide-by-zero guard, uske boundary tests likhunga, `generateFinancialSummary` mein call karunga, output interface mein field add karunga, dashboard par render karunga. Engine isi liye modular hai.

C. Scheme Matcher (Q31-Q45)

Q31. Scheme matching algorithm explain karo.

`matchSchemes(profile, schemes[])` har scheme par map karta hai. Har scheme ke `eligibility_rules` JSONB se 6 possible checks — income, category, sector, gender, state, aur empty-rules case. Har check pass/fail par ek human-readable ✓/✗ reason push hoti hai. Koi ek bhi fail ho to `eligible=false`, par loop rukta nahi — baaki reasons bhi collect hote hain. Aakhir mein eligible-first sort.

Q32. Complexity kya hai?

Map $O(n)$ jahan n = schemes, har scheme par constant checks (max 6) aur chhote arrays par `includes()`. Sort $O(n \log n)$. Total **$O(n \log n)$** . $n=15$ hai to practically instant.

Q33. Eligibility rules JSONB mein kyun, alag columns mein kyun nahi?

Flexibility. Har scheme ke alag rules hote hain — kisi mein gender, kisi mein state, kisi mein sirf income. Alag columns banate to 20 nullable columns hote aur nayi rule type par migration chahiye hoti. JSONB ke saath nayi scheme = ek `INSERT`. Code redeploy nahi, migration nahi. Non-technical person Supabase UI se kar sakta hai.

Q34. JSONB aur JSON mein Postgres mein kya farak?

JSON text as-is store karta hai (whitespace, key order preserve). JSONB binary parsed form mein — thoda slow insert, par bahut fast query aur GIN indexing support. Humne JSONB chuna kyunki rules read zyada hote hain, write kam.

Q35. Ineligible schemes ke liye bhi reasons kyun? Early exit kyun nahi?

Ye humara core differentiator hai — **explainability**. Sirf "not eligible" bolne se user kuch nahi kar sakta. Par "aap eligible nahi ho kyunki ye scheme SC/ST ke liye reserved hai" bolne se user samajh sakta hai kya karna hai. Early exit se ye information kho jati.

Q36. AND logic kyun, OR kyun nahi?

Government eligibility conjunctive hoti hai — Stand-Up India ke liye SC/ST **aur** correct sector **aur** loan range, sab chahiye. Ek bhi fail = ineligible. OR logic legally galat result deta.

Q37. Lowercase comparison kyun?

Data multiple sources se aata hai — voice transcription, form, facilitator entry, DB seed. "SC"/"sc"/"Sc" sab possible. Dono sides `.toLowerCase()` karke compare karte hain taaki false negative na ho.

Q38. `rules.income_max !== undefined` kyun, sirf `if (rules.income_max)` kyun nahi?

Kyunki `income_max: 0` valid value ho sakti hai, par JS mein 0 falsy hai — truthy check usko skip kar deta. `!== undefined` explicitly presence check karta hai.

Q39. `toLocaleString('en-IN')` kyun?

Indian numbering system. 540000 → "5,40,000" (lakh grouping), na ki "540,000". Rural user ke liye readability mein bada farak. Chhoti detail par user-centric design dikhati hai.

Q40. Sort mein `return 0` kyun?

ES2019 se JavaScript ka `Array.sort` **stable** hai. `return 0` se original order preserve hota hai — same-eligibility schemes apna DB order maintain karti hain, UI predictable rehta hai.

Q41. Nayi scheme kaise add karoge?

Ek SQL INSERT `schemes` table mein — name, description, benefit_summary, eligibility_rules JSONB, application_link. Koi code change nahi. Agar bilkul naya rule type chahiye (jaise "age_max"), tab schemeMatcher mein ek naya check block add karna padega.

Q42. Kai schemes eligible ho to prioritize kaise karoge?

Abhi sirf eligible-first sort hai, ranking nahi. Future: benefit amount, application complexity, approval probability, aur user ke current stage ke hisaab se scoring. Jaise ₹50,000 chahiye to Mudra Shishu upar, Stand-Up India (₹10 lakh min) neeche.

Q43. Scheme rules purane ho jayein to?

Real limitation hai. Abhi manual seed data. Mitigation: admin panel + quarterly review workflow, long-term government open-data API sync. Har scheme ka official `application_link` bhi rakha hai taaki user authoritative source par verify kar sake.

Q44. Matcher galat output de to user ko nuksan ho sakta hai?

Teen safeguards: (1) har decision ka reason dikhate hain taaki user verify kar sake, (2) official government link dete hain, (3) kahin bhi "loan milega" guarantee nahi karte — sirf "aap criteria meet karte hain". Final decision bank/government ka hai.

Q45. Scheme matcher ke test cases kaise design kiye?

Real personas — SC low-revenue, female ST, high-revenue general male, agriculture sector, aur "koi category nahi" case. Positive matches aur **negative cases dono** (income exceed, category mismatch). Plus sorting behavior aur reason string format verify kiya.

D. Database & Security (Q46-Q65)

Q46. Kitni tables hain aur kya kya?

8 tables: `users`, `business_profiles`, `schemes`, `financial_plans`, `ledger_entries`, `conversations`, `facilitators_entrepreneurs`, aur `business_guides` (migration 004 mein add hui).

Q47. Row Level Security kya hai?

Postgres ka feature jisme har table par policies lagti hain jo **database level** par decide karti hain ki kaunsi rows kaun dekh/badal sakta hai. Supabase har request ke JWT se `auth.uid()` nikaalta hai. Fayda: agar application code mein bug bhi ho, tab bhi user doosre ka data nahi padh sakta — defense in depth.

Q48. USING aur WITH CHECK mein farak?

`USING` existing rows par filter hai — "kaunsi rows dikhengi" (SELECT/UPDATE/DELETE). `WITH CHECK` new ya modified row par validation hai — "kaunsi row likhi ja sakti hai" (INSERT/UPDATE). UPDATE par dono chahiye: `USING` se decide ki kaunsi row edit kar sakte ho, `WITH CHECK` se rokta hai ki edit karke `user_id` kisi aur ka na bana do — ownership hijack se bachav.

Q49. Facilitator entrepreneur ka data edit kar sakta hai?

Nahi. Policies mein facilitator ko sirf SELECT par dual-condition diya hai. INSERT/UPDATE/DELETE policies mein sirf `auth.uid() = user_id` hai. Read-only supervision — ye deliberate design hai, facilitator kisi ka financial data manipulate na kar sake.

Q50. Conversations table facilitator ko kyun nahi dikhti?

Privacy. Conversation mein personal chat history hoti hai. Business metrics supervision ke liye theek hai, par private conversations nahi. Isliye conversations ki saari policies sirf `auth.uid() = user_id` hain.

Q51. Service role key RLS bypass karti hai — ye security risk nahi?

Haan, ye trade-off hai aur hum ise consciously manage karte hain. Zaroori isliye hai kyunki WhatsApp/SMS user ke paas koi browser session ya JWT nahi hota — sirf phone number hai; RLS ke through wo kuch nahi kar sakta. Mitigations: key par `NEXT_PUBLIC_` prefix nahi hai to wo kabhi browser bundle mein nahi jati; client config mein `persistSession: false, autoRefreshToken: false` set hai; aur jahan "kaun logged in hai" jaanna hota hai wahan hum `createSupabaseRouteClient()` use karte hain jo RLS respect karta hai. Ideal production mein service-role usage aur kam karna chahiye.

Q52. Paise ke liye NUMERIC kyun, FLOAT kyun nahi?

FLOAT/DOUBLE binary floating point hain — 0.1 exactly represent nahi hota, isliye rounding errors accumulate hote hain. Financial data mein ye unacceptable hai. `NUMERIC(12,2)` exact decimal arithmetic deta hai — 12 total digits, 2 decimal places, matlab ₹99,99,99,999.99 tak.

Q53. TIMESTAMPTZ aur TIMESTAMP mein farak?

`TIMESTAMPTZ` timezone-aware hai — internally UTC mein store karta hai aur client timezone mein convert karta hai. `TIMESTAMP` timezone-naive hai. Humne `TIMESTAMPTZ` chuna kyunki users alag timezones mein ho sakte hain aur server UTC mein chalta hai.

Q54. UUID primary key kyun, auto-increment integer kyun nahi?

Teen reasons: (1) UUIDs guessable nahi hote — sequential IDs se koi `/dashboard?user_id=1,2,3` try kar sakta hai (enumeration attack), (2) distributed systems mein bina coordination ke generate ho sakte hain, (3) Supabase Auth khud UUIDs use karta hai to `auth.users.id` se match karana aasan hai.

Q55. CHECK constraints kyun lagaye?

Database level par data integrity. `role IN ('entrepreneur','facilitator','admin')`, `entry_type IN ('income','expense')`, `gender IN ('male','female','other')`. Agar application code mein bug ho aur galat value bheje, DB usko reject kar dega. Humein iska real fayda mila — gender constraint ne ek bug pakda tha.

Q56. Wo gender wala bug kya tha?

Onboarding form gender collect nahi karta, to code default `'any'` bhej raha tha. Par DB par `CHECK (gender IN ('male','female','other'))` tha, to har onboarding submission "Failed to save business profile" se fail ho rahi thi. Fix: default `'other'` kar diya. Ye acchi misaal hai ki DB constraints application bugs pakadte hain.

Q57. ON DELETE CASCADE kya karta hai?

Parent row delete hone par child rows automatically delete ho jati hain. Humne saare `user_id` foreign keys par lagaya hai — user delete karo to uske profile, plans, ledger, conversations sab delete. Ye GDPR/DPDP ke "right to erasure" ke liye important hai.

Q58. Indexes kahan lagaye aur kyun?

`users.phone` — har WhatsApp/SMS message phone se lookup karta hai, sabse frequent query. Saare foreign keys (`user_id`) par — warna har join full table scan karta. `ledger_entries.created_at` — dashboard 15-din ka chart banata hai, date range query fast chahiye. `business_guides.created_at DESC` — latest-first listing.

Q59. Migrations idempotent kaise banaye?

Saare statements `CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`, `CREATE EXTENSION IF NOT EXISTS` use karte hain. Matlab migration dobara chalane par error nahi aayega — safe re-run. Ye deployment mein bahut helpful hai.

Q60. financial_plans mein matched_scheme_ids array aur plan_json dono kyun?

Intentional denormalization. Array (`UUID[]`) fast querying ke liye hai — "kitne users PMEGP ke eligible hain?" jaisi analytics. JSONB complete audit trail ke liye — har reason string, timestamp, profile snapshot. Alag purposes hain.

Q61. Migration 003 kyun missing hai?

Numbering gap hai — 001, 002, phir 004. Wo migration probably develop ki gayi thi phir consolidate/drop ho gayi. Root `schema.sql` poora consolidated schema rakhta hai jo single-shot setup ke liye use hota hai. *Honest answer dena — gap explain karna better hai bajaye kuch bana kar bolne ke.*

Q62. upsert kya hai aur onConflict kya karta hai?

Upsert = insert-or-update. Agar row exist nahi karti to insert, karti hai to update. `onConflict: 'user_id'` batata hai ki conflict kis column par detect karna hai. Humne `business_profiles` par `user_id`, `users` par `id`, aur `facilitators_entrepreneurs` par composite `'facilitator_id,entrepreneur_id'` use kiya hai.

Q63. PGRST116 error code kya hai?

PostgREST ka code for "no rows found" jab `.single()` use karo. Ye actual error nahi hai — expected case hai (naya user). Isliye orchestrator mein hum specifically check karte hain: `if (userError && userError.code !== 'PGRST116')` — sirf real errors log karo.

Q64. Schemes table par INSERT policy kyun nahi banayi?

Deliberate. Sirf SELECT policy hai (`USING true` for authenticated aur anon). Jab kisi operation ke liye policy nahi hoti, RLS default-deny karta hai. Matlab koi bhi normal user scheme add/edit/delete nahi kar sakta — sirf admin service-role key se. Government scheme data ki integrity protect karta hai.

Q65. Data breach ho jaye to kitna PII expose hoga?

Name, phone/email, district/state, aur business financial estimates. Humne deliberately Aadhaar number, bank account number, ya exact address store nahi kiya — **data minimization principle**. Aur Gemini ko to PII bilkul nahi bhejte. Yahi DPDP Act ka core requirement hai: sirf wahi data collect karo jo purpose ke liye zaroori hai.

E. AI / LLM Integration (Q66-Q82)

Q66. Kaunsa LLM use kiya aur kyun?

Google Gemini 2.5 Flash. Reasons: (1) Flash variant fast aur sasta hai — humare tasks (extraction, rephrasing) complex reasoning nahi maangte, (2) `responseMimeType: 'application/json'` se structured output guaranteed milta hai, (3) Indian languages par accha performance, (4) generous free tier — student project ke liye important.

Q67. LLM se numbers kyun nahi calculate karwate?

Teen reasons. Ek — LLMs probabilistic hain, wo arithmetic mein hallucinate kar sakte hain. Do — financial advice mein galat number life-changing damage kar sakta hai; agar hum galat break-even bata dein aur user uske hisaab se loan le le, to responsibility humari hai. Teen — deterministic output **audit aur prove** kiya ja sakta hai; LLM ka nahi. Isliye architecture mein clear separation hai: math code karta hai, language LLM.

Q68. Prompt engineering kaise ki?

Teen techniques. (1) **System instructions with explicit negative constraints** — "Do NOT change any numbers", "NEVER guess or invent values", "Return only plain text without markdown". (2) **Structured output mode** — `responseMimeType: 'application/json'` plus exact schema prompt mein. (3) **Low temperature** — 0.1 extraction ke liye, 0.2 roadmap, 0.3 rephrasing. Creativity kam, consistency zyada.

Q69. Temperature kya hai aur alag-alag kyun rakhi?

Temperature output randomness control karta hai — 0 = most deterministic, 1+ = creative. Extraction (0.1) mein bilkul creativity nahi chahiye, sirf accurate parsing. Roadmap (0.2) mein thodi variation chalegi. Rephrasing (0.3) mein thoda natural language flow chahiye. Task ke hisaab se tune kiya.

Q70. Gemini down ho jaye to kya hoga?

Kuch nahi tootega. Teeno jagah fallback hai: (1) `/api/plan/generate` — `llmSummaryText` ki initial value hi deterministic English explanation hai, catch block sirf log karta hai. (2) `/api/onboarding/parse` — regex-based `fallbackParser()` chalta hai, aur response mein `source: 'fallback'` return hota hai. (3) `/api/business-guide/generate` — poora Hindi deterministic roadmap hardcoded hai jo user ke actual numbers use karta hai. User ko kabhi error nahi dikhta, bas quality thodi kam ho jati hai.

Q71. LLM ko PII kyun nahi bhejte? Exactly kya bhejte ho?

Sirf numeric financial summary (revenue, expense, margin%, risk level, break-even) aur matched scheme names. Kabhi nahi: naam, phone, email, exact address/village, social category, gender. Reason: (a) DPDP Act data minimization, (b) third-party service par personal data bhejna consent scope badha deta hai, (c) rural users ki caste/gender information especially sensitive hai.

Q72. Fir LLM Hindi mein personalized kaise likhta hai bina naam ke?

Personalization naam se nahi, **situation** se aati hai. "Aapka munafa 38% hai, jo achha hai. Aapke liye Mudra Shishu loan upyukt ho sakta hai." Ye already highly personalized hai. Naam add karne se koi advisory value nahi badhti — sirf privacy risk badhta hai.

Q73. Business guide mein 5 stages fix kyun rakhe?

Do reasons. Ek — ye ek proven micro-enterprise growth framework hai: pehle cost kam karo (immediate cash), phir value add karo (margin), phir direct market access (revenue), phir formalize karo (credit access unlock), phir scale. Sequence logical hai. Do — fixed structure se UI predictable rehta hai aur LLM output validate kiya ja sakta hai.

Q74. LLM ne stage ka naam badal diya ya 4 stages diye to?

Code handle karta hai. Pehle `Array.isArray(parsed) && parsed.length === 5` check hota hai — nahi to poora fallback roadmap. Agar length sahi hai, to hum `stage: FIXED_STAGES[idx]` se LLM ka stage name **overwrite** kar dete hain. Har field par `|| default` hai. Matlab UI ko hamesha exactly 5 valid stages milte hain.

Q75. Prompt injection ka khatra hai?

Haan, theoretically. User apne challenge_text mein likh sakta hai "ignore previous instructions". Mitigation abhi ye hai ki (a) LLM ka output kabhi directly execute nahi hota — sirf display hota hai, (b) numbers LLM se nahi aate, to injection se financial damage nahi ho sakta, (c) output schema-validated hai. Par proper input sanitization aur output content filtering add karna future work hai. *Ye honestly acknowledge karna better hai.*

Q76. LLM cost kaise control karoge scale par?

Flash model already sabse sasta variant hai. Aage: (1) same profile ka summary cache karo — profile na badle to dobara generate mat karo, (2) batch processing, (3) rate limiting per user, (4) fallback ko default banao aur LLM ko opt-in feature.

Q77. OCR kaise kaam karta hai?

Tesseract.js — ek WebAssembly port hai Google ke Tesseract OCR engine ka, pure JavaScript mein chalta hai. Hum `'eng+hin'` dono language packs load karte hain. Image ko Buffer mein convert karke `Tesseract.recognize()` call karte hain, jo raw text return karta hai. Phir 3 regex patterns se amounts aur descriptions extract karte hain.

Q78. OCR server par kyun chalte ho, client par kyun nahi?

Do reasons: (1) Rural users ke phones kamzor hote hain — WASM OCR unke browser ko hang kar sakta hai, (2) language data files (~10MB+) har baar download karni padti. Server par ek baar load hokar reuse hoti hain. Trade-off: server par CPU cost badhta hai.

Q79. OCR ki accuracy kitni hai?

Printed bills par decent, handwritten Devanagari par kamzor. Isliye humne design mein **human-in-the-loop** rakha hai — saari OCR entries `confirmed: false` ke saath save hoti hain aur user dashboard par confirm karta hai tabhi wo real financial data banti hai. OCR ko trust nahi karte, sirf data-entry effort kam karte hain.

Q80. OCR saari entries ko 'income' kyun mark karta hai?

Kyunki bill ki image dekh kar ye batana ki wo income hai ya expense, semantic understanding chahiye jo Tesseract ke paas nahi hai. Humne default 'income' rakha aur user correction par chhoda. Future improvement: keyword classifier (kharid/purchase → expense, bikri/sale → income) ya Gemini Vision se classification.

Q81. Voice recognition kaise kaam karta hai?

Browser ka native Web Speech API. `SpeechRecognition` (Chrome mein `webkitSpeechRecognition`) — `lang='hi-IN'` set kiya hai jo Hindi aur Hinglish dono theek handle karta hai, `continuous=false` (ek sentence

sun kar ruk jao), `interimResults=true` (live transcript dikhao). Output text phir `/api/onboarding/parse` par jata hai Gemini extraction ke liye. Aur TTS ke liye `speechSynthesis` API se questions bol kar sunate hain.

Q82. Firefox Web Speech API support nahi karta — to?

Isliye `/onboarding` page par browser detection hai: `'webkitSpeechRecognition' in window || 'SpeechRecognition' in window`. Support ho to voice modal auto-open hota hai, na ho to seedha 8-step text form dikhta hai — jo fully functional accessible fallback hai. Aur voice modal mein bhi hamesha ek manual text input box visible rehta hai.

F. Next.js / Frontend (Q83-Q100)

Q83. Next.js kyun chuna, plain React kyun nahi?

Teen reasons: (1) Ek hi codebase mein frontend aur backend — alag Express server maintain karne ki zaroorat nahi, (2) Server Components se DB directly query kar sakte hain bina API layer ke — dashboard isi tarah bana hai, (3) File-based routing, built-in optimization (images, fonts, code splitting), aur Vercel par zero-config deployment.

Q84. Server Component aur Client Component mein farak?

Server Component server par render hota hai, uska JavaScript browser tak nahi jata. Wo DB query kar sakta hai, secrets use kar sakta hai, par `useState` / `useEffect` / event handlers nahi use kar sakta. Client Component (`'use client'`) browser mein hydrate hota hai — interactive ho sakta hai par DB directly access nahi kar sakta.

Q85. Dashboard Server Component kyun banaya?

Kyunki wo pure data display hai. Server Component hone se: (a) koi loading spinner nahi — HTML already data ke saath aata hai, (b) service-role key safely use kar sakte hain, (c) client bundle chhota rehta hai, (d) SEO/first-paint better. Interactivity sirf LogoutButton mein chahiye thi, jo alag Client Component hai.

Q86. `export const dynamic = 'force-dynamic'` kya karta hai?

Next.js ko batata hai ki ye page har request par freshly render karo — static generate ya cache mat karo. Dashboard ke liye zaroori hai kyunki har user ka data alag hai aur real-time hona chahiye.

Q87. Next.js 16 mein "proxy" kya hai?

Next.js 16 mein "Middleware" file convention ka naam badal kar "Proxy" ho gaya hai — same runtime, same purpose. File `src/proxy.ts` hai aur exported function `proxy()`. Pehle ye `middleware.ts` / `middleware()` tha.

Q88. Aapka proxy kya karta hai?

Har matched request par `supabase.auth.getUser()` call karta hai. Iska side effect ye hai ki agar access token expire ho gaya par refresh token valid hai, to naya token aa jata hai aur cookie rewrite ho jati hai. Iske bina users ko random logouts milte. Poora block try/catch mein hai kyunki ye har request par chalta hai — agar Supabase Auth 2 second down ho aur ye throw kare, to poori site 500 de deti.

Q89. Matcher regex kya karta hai?

`['/(?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp|ico)$).*(.*)']` — negative lookahead se static assets, images, aur favicon skip karta hai. Sirf actual page/API requests par proxy chalta hai. Performance optimization.

Q90. PKCE flow kya hai?

Proof Key for Code Exchange (RFC 7636). Public clients — jaise browser apps — `client_secret` safely store nahi kar sakte. PKCE mein har login par random `code_verifier` generate hota hai; uska SHA-256 hash (`code_challenge`) authorization request ke saath jata hai. Baad mein code exchange karte waqt original verifier bhejna padta hai. Agar attacker authorization code chura bhi le, wo bina verifier ke session nahi bana sakta.

Q91. Session cookies mein kyun store kiya, localStorage mein kyun nahi?

Kyunki localStorage sirf browser mein hota hai — server usse padh nahi sakta. To Server Components ko pata hi nahi chalta ki user logged in hai. Cookies har request ke saath automatically server tak jati hain. Isliye humne @supabase/ssr ka createBrowserClient use kiya, plain supabase-js nahi. Aur localStorage hone par Supabase default implicit flow use karta, PKCE nahi — jisse /auth/callback ka code exchange hi toot jata.

Q92. Teen Supabase clients kyun hain?

supabaseClient (browser, anon key, RLS on) — Client Components ke liye. supabaseServer (service-role key, RLS bypass) — API routes aur admin operations ke liye. createSupabaseRouteClient() (anon key + request cookies, RLS on) — server par jab jaanna ho ki kaun logged in hai. Har ek ka clear purpose hai.

Q93. Zod kyun use kiya jab TypeScript already hai?

TypeScript sirf **compile-time** check karta hai — runtime par types erase ho jate hain. Network se aaya JSON kuch bhi ho sakta hai. Zod **runtime** validation karta hai aur structured errors deta hai (error.flatten()), jisse client ko field-wise error messages bhej sakte hain. Aur Zod schema se TypeScript type infer bhi ho jata hai — single source of truth.

Q94. Zod ka .refine() kya karta hai aapke code mein?

Custom validation rule add karta hai. Humne DPDP consent par lagaya hai: consent_given: z.boolean().refine(val => val === true, { message: 'Explicit DPDP Act consent is required...' }). Matlab false bhejne par request validation stage par hi 400 se reject ho jati hai — DB tak pahunchti hi nahi. Legal requirement ko type system mein enforce kar diya.

Q95. React Context kyun use kiya language ke liye?

Language state ko har component tak prop drilling ke bina pahunchana tha. LanguageProvider root layout mein wrap hai; koi bhi client component useLanguage() se t(), language, aur toggleLanguage() access kar sakta hai. Redux jaisi library is scale par overkill hoti.

Q96. useCallback aur useRef kahan use kiye aur kyun?

VoiceOnboardingModal mein. useRef SpeechRecognition instance ke liye — re-render par naya object na bane aur ongoing recognition na tute. useCallback speakText, startListening, stopListening, changeStep par — stable function references taaki useEffect dependency arrays mein infinite loop na ho.

Q97. Functional state update kyun use kiya?

setFormData(prev => ({...prev, name})) — stale closure bug se bachne ke liye. Agar seedha setFormData({...formData, name}) likhein to async operations mein purana formData capture ho sakta hai aur pichhle steps ka data kho jata.

Q98. useTransition kahan use kiya?

/dashboard/schemes page ke "Recheck" button mein. Wo schemes dobara fetch karke rematch karta hai. useTransition se ye update non-blocking hoti hai — UI freeze nahi hota aur isPending se loading state dikha sakte hain.

Q99. Tailwind CSS kyun?

Utility-first approach se CSS files maintain nahi karni padti, aur unused CSS automatically purge ho jata hai — final bundle chhota. Design system ke colors/spacing consistent rehte hain. v4 mein config JS file ki jagah CSS

mein `@import "tailwindcss"` se hota hai.

Q100. Fonts kaise handle kiye?

Do tarah se. Google Fonts (Inter, Playfair Display, Poppins, Material Symbols) `layout.tsx` ke `<head>` mein `<link>` se, with `preconnect` hints. Aur custom fonts (Bodoni Moda, EB Garamond, Space Grotesk) self-hosted `.ttf` files `/public/fonts` mein, `@font-face` with `font-display: swap` — taaki text turant dikhe fallback font mein aur baad mein swap ho jaye.

G. Integrations & System Design (Q101-Q118)

Q101. WhatsApp webhook verification kaise hota hai?

Meta ek GET request bhejta hai teen query params ke saath: `hub.mode=subscribe`, `hub.verify_token` (jo humne Meta dashboard mein set kiya tha), aur `hub.challenge` (random string). Hum verify token match karte hain aur challenge ko **as-is plain text** mein 200 ke saath wapas bhejte hain. JSON return karna galat hai — verification fail ho jayega. Match na ho to 403.

Q102. WhatsApp POST mein fire-and-forget pattern kyun?

Meta 20 second ke andar 200 OK expect karta hai. Agar hum orchestrator + Gemini + DB sab await karein to zyada time lag sakta hai, aur Meta message retry karega — duplicate processing ho jayegi. Isliye `processWhatsAppMessages(body).catch(...)` bina await ke chalte hain aur turant 200 return karte hain.

Q103. Fire-and-forget mein serverless function toh khatam ho sakta hai processing complete hone se pehle?

Haan, ye real risk hai — ye pattern ki known limitation hai. Vercel serverless function response bhejne ke baad terminate ho sakta hai. Proper solution ek job queue hai (Inngest, QStash, ya Vercel `waitUntil`) jo background work guarantee kare. MVP mein practically kaam kar raha hai kyunki processing fast hai, par production ke liye ye fix karna chahiye. *Ye batana ki aapko limitation pata hai, bada plus point hai.*

Q104. WhatsApp aur SMS handling mein architectural farak kya hai?

WhatsApp **asynchronous** hai — webhook 200 deta hai, phir hum separately Graph API par POST karke reply bhejte hain. SMS **synchronous** hai — reply usi HTTP response ke body mein TwiML XML ke roop mein jata hai. Dono ke liye orchestrator same hai, sirf transport layer alag hai.

Q105. TwiML kya hai?

Twilio Markup Language — XML format jo Twilio ko batata hai kya karna hai. SMS ke liye: `<Response><Message>text</Message></Response>`. Content-Type `text/xml` hona chahiye.

Q106. XML escaping kyun zaroori hai?

Injection prevention. Agar user ka naam `</Message><Message>spam` ho aur hum use as-is XML mein daal dein, to attacker Twilio ko extra SMS bhejne par majboor kar sakta hai. Humara `twimlResponse()` 5 characters escape karta hai: `&`, `<`, `>`, `"`, `'`. **Order important hai** — `&` sabse pehle, warna `<` dobara escape hokar `&lt;` ban jata.

Q107. State machine kya hai aur aapne kyun use kiya?

Finite State Machine — system kisi ek definite state mein hota hai aur inputs se defined transitions se doosre state mein jata hai. Conversation ke liye perfect fit hai kyunki har step par hum exactly jaante hain ki agla sawaal kya hai aur jawab kaise validate karna hai. Alternative — free-form LLM conversation — unpredictable hoti aur data quality guarantee nahi kar sakti.

Q108. Aapke FSM ke states kaunse hain?

6 states: `idle` → `awaiting_sector` → `awaiting_district` → `awaiting_revenue` → `awaiting_expenses` → `awaiting_loans` → `complete`. Invalid input par state change nahi hota — same state mein rehkar retry message bhejta hai. `complete` state mein "PLAN" command sunta hai.

Q109. Conversation state DB mein kyun persist karte ho, memory mein kyun nahi?

Kyunki Vercel par har API call ek **stateless serverless invocation** hai — do messages ke beech koi shared memory nahi, process hi alag ho sakta hai. DB mein persist karne se: (a) horizontally scalable — koi bhi instance koi bhi message handle kar sakta hai, (b) crash-resilient — server restart par user ka progress nahi jata, (c) auditable — poori history DB mein.

Q110. Same user WhatsApp aur SMS dono use kare to?

Conversation query `user_id` aur `channel` dono se filter karti hai. Matlab dono channels ki alag conversation rows hongi, alag state. Par user aur business profile same rahenge (phone se identify hote hain). Ye design intentional hai — channel-specific progress track hota hai.

Q111. parseAmount aur parseYesNo kaise kaam karte hain?

`parseAmount` : ₹, commas, whitespace strip karke `parseFloat`, phir NaN ya negative reject. `parseYesNo` : 9 yes-variants aur 7 no-variants ki list — Devanagari (हाँ, नहीं), Roman Hindi (haan, nahi), English (yes, no), aur shortcuts (y, n, 1, 0). `startsWith` use karte hain taaki "haan ji bilkul" bhi match kare.

Q112. Keepalive endpoint kyun banaya?

Supabase free tier project inactivity ke baad pause kar deta hai — phir pehla request bahut slow hota hai (cold start). UptimeRobot har 5 minute `/api/keepalive` hit karta hai jo DB par ek halka count query chalata hai. `head: true` use kiya hai taaki sirf count aaye, rows nahi. Aur `Cache-Control: no-store` header hai warna CDN cache kar leta aur DB hit hi nahi hoti.

Q113. Keepalive DB down hone par bhi 200 kyun return karta hai?

Design choice. Body mein `status: 'degraded'` aata hai. Agar 500 return karein to UptimeRobot "site down" alert bhej dega jabki actually sirf DB slow hai aur baaki site chal rahi hai. Monitoring granularity better rehti hai.

Q114. Facilitator ka role kya hai?

Field worker — NGO worker, bank mitra, ya SHG coordinator. Wo un entrepreneurs ko onboard karta hai jo khud nahi kar sakte (no phone, no literacy). Uske paas `/facilitator` portal hai jahan wo apne linked entrepreneurs ka profile aur latest plan dekh sakta hai — **read-only**. Edit nahi kar sakta.

Q115. Facilitator ka access RLS mein kaise implement hua?

Har relevant table ki SELECT policy mein dual condition hai: `auth.uid() = user_id OR user_id IN (SELECT entrepreneur_id FROM facilitators_entrepreneurs WHERE facilitator_id = auth.uid())`. Matlab apna data ya apne linked entrepreneurs ka. INSERT/UPDATE policies mein sirf pehli condition hai.

Q116. System offline kaam kar sakta hai?

Abhi nahi — web app ko internet chahiye. Par SMS channel effectively "offline-tolerant" hai kyunki SMS cellular network par chalta hai, internet par nahi. Future scope mein offline-first PWA hai jisme service worker se data cache aur background sync ho.

Q117. Ek hi phone number par do log business chalate hon to?

Abhi `users.phone` UNIQUE hai aur ek user ka ek `business_profiles` row hai (`onConflict: 'user_id'`). To ek phone = ek business. Multi-business support ke liye schema change chahiye — `business_profiles` ko one-to-many banana aur UI mein business selector add karna.

Q118. System architecture mein single point of failure kya hai?

Supabase. Agar DB down ho to authentication, profile, plans, sab affect hote hain. Mitigations jo humne kiye: dashboard par 4-level fallback (demo user tak), `FALLBACK_SCHEMES` hardcoded array, aur proxy ka try/catch. Proper solution: read replicas, caching layer (Redis), aur graceful degradation UI.

H. Tough / Critical Questions (Q119-Q135)

Ye wo questions hain jo examiner aapko *ghera* mein lene ke liye puchta hai. Defensive mat bano — honest aur reasoned answer do.

Q119. Ye to bas ek CRUD app hai with some formulas. Innovation kya hai?

Innovation teen jagah hai. Ek — **architectural**: hybrid deterministic-plus-LLM design jahan numbers code se aate hain aur language LLM se, with full fallback chain. Ye financial AI ka safe pattern hai jo aksar log ulta karte hain. Do — **accessibility**: same brain teen channels par, including SMS jo bina internet ke chalta hai; ye zero-bandwidth inclusion hai. Teen — **explainability**: har eligibility decision ka reason string, ineligible cases ke liye bhi — jo user ko actionable information deta hai, sirf verdict nahi.

Q120. Break-even calculation to galat hai — aap variable cost 0 maan rahe ho.

Aap bilkul sahi pakad rahe hain, aur ye humne code ke comment mein khud document kiya hai. Onboarding par per-unit data available nahi hota. Isliye humne **output ko units ki tarah present kiya hi nahi** — user ko hum revenue target batate hain: "₹11,000 monthly revenue maintain karna hoga costs cover karne ke liye", jo mathematically correct statement hai. True unit-based break-even ke liye per-unit price aur variable cost chahiye, jo optional advanced onboarding step ke roop mein next iteration mein add karenge.

Q121. Aapki 0.7/0.9 thresholds arbitrary hain.

Wo heuristics hain, arbitrary nahi — microfinance practice se derived. Par aapka point valid hai ki wo empirically validated nahi hain. Humne unhe deliberately **named constants ke roop mein ek hi pure function mein** rakha hai taaki real data milne par ek jagah calibrate ho sakein, aur ideally sector-wise. Abhi conservative aur explainable rakha hai — user ko reason dikhta hai, sirf label nahi.

Q122. Service role key use karke aapne apni hi RLS bekaar kar di.

API routes mein haan, RLS bypass hoti hai — ye zaroori hai kyunki WhatsApp/SMS users ke paas JWT session nahi hota. Par RLS bekaar nahi hai: (a) client-side queries (dashboard schemes page, business guide page) `supabaseClient` use karti hain jo anon key par RLS respect karta hai, (b) agar kabhi anon key leak ho ya koi directly Supabase REST API hit kare, RLS unhe rokegi, (c) `createSupabaseRouteClient()` bhi RLS respect karta hai aur auth-sensitive jagah use hota hai. RLS defense-in-depth ki ek layer hai, akeli layer nahi.

Q123. Aapne API routes ke tests kyun nahi likhe?

Honest answer: time constraint aur prioritization. Humne testing effort wahan concentrate kiya jahan galti sabse mehngi hai — financial calculations aur eligibility decisions, jo 100% tested hain. API routes mostly orchestration hain — validate, fetch, call engine, save. Un engines ka logic already tested hai. Par aap sahi hain ki integration tests missing hain; next step MSW-based mocking ke saath route tests hain.

Q124. Gemini API key aapke paas free tier ka hai — production mein kya karoge?

Paid tier par jayenge, par usse pehle cost optimization: same profile ke liye summary cache karna, LLM calls ko optional banana (deterministic fallback already exist karta hai), aur per-user rate limiting. Aur agar cost prohibitive ho to hum **LLM ke bina bhi fully functional hain** — ye humari architecture ki strength hai.

Q125. User ko galat scheme suggest ho gayi aur uska loan reject ho gaya to zimmedar kaun?

Humara system kahin bhi "aapko loan milega" guarantee nahi karta. Hum sirf ye batate hain ki "aap is scheme ke published criteria meet karte hain" — with the exact reasons dikhaye gaye. Final eligibility determination bank ya implementing agency karti hai. Har scheme ka official government link bhi dete hain taaki user authoritative source par verify kar sake. Ye humne deliberately design kiya hai — advisory, not authority.

Q126. Rural user ko English reasons kaise samajh aayenge? Aapke ✓/X strings English mein hain.

Valid gap. Abhi scheme matcher ke reason strings English mein generate hote hain. Mitigation jo hai: (a) `/api/plan/generate` mein Gemini poore summary ko user ki language mein rephrase karta hai, (b) schemes page par documents aur ministry names Hindi+English dono mein hain, (c) WhatsApp/SMS pe orchestrator ke saare messages Hindi mein hain. Par matcher ke reasons ko bhi i18n karna chahiye — ye clear next task hai.

Q127. Aapne bola LLM numbers nahi badalta — ye guarantee kaise karte ho?

Guarantee nahi kar sakte — ye prompt-level constraint hai, hard constraint nahi. Isliye humne teen layers rakhe: (a) system instruction mein explicit prohibition, (b) temperature 0.3 par, (c) **original deterministic explanation bhi compute hota hai aur wahi fallback hai**. Aur critically — jo numbers **DB mein save hote hain** (`margin_percent`, `break_even_units`, `plan_json`) wo engine se aate hain, LLM se nahi. LLM ka output sirf `summary_text` field mein jata hai. To agar LLM number badal bhi de, system ka structured data correct rehta hai. Ek proper improvement hoga: LLM output mein numbers ko regex se extract karke original se verify karna.

Q128. Dashboard mein demo user fallback hai — ye to security issue hai.

Aap sahi hain. Level 3 fallback ("DB ka latest user") production mein **privacy issue** hai — koi unauthenticated visitor kisi aur ka data dekh sakta hai. Ye purely demo-resilience ke liye add kiya gaya tha taaki SIH judging mein network hiccup par blank page na dikhe. Production checklist mein level 3 aur 4 remove karke proper `redirect('/login')` lagana pehla item hai.

Q129. groq-sdk install hai par use nahi hota — kyun?

Legacy dependency. Initially Groq (fast Llama inference) evaluate kiya tha, phir Gemini par settle kiye kyunki uska structured JSON output mode aur Indian language performance better tha. `groq-sdk` remove kar dena chahiye — dependency hygiene ka issue hai.

Q130. Aapne migration 003 kahan gayi?

Numbering mein gap hai — 001, 002, 004. Wo migration development ke dauran banayi gayi thi aur phir consolidate ya drop ho gayi. Root `schema.sql` poora consolidated schema rakhta hai jo fresh setup ke liye single-shot chalaya ja sakta hai. Migration numbering clean karni chahiye.

Q131. Ek scheme add karne ke liye SQL likhni padti hai — non-technical team kaise manage karega?

Abhi Supabase ka built-in table editor UI use kar sakte hain — JSONB field bhi wahan edit ho jata hai. Par proper solution ek admin panel hai with form-based scheme editor aur validation. Ye future scope mein hai. JSONB design ne ye future-proof to bana diya hai ki wo panel banane par code change nahi chahiye.

Q132. Aapka system biased ho sakta hai? Kaise ensure karte ho fairness?

Do angles. Ek — **eligibility decisions bilkul rule-based hain**, LLM-based nahi. To model bias eligibility par asar nahi daalta; rules wahi hain jo government ne publish kiye hain. Do — **LLM sirf phrasing karta hai aur usse category/gender data bheja hi nahi jata**, to wo tone mein bias nahi daal sakta based on caste ya gender. Ye data minimization ka ek fairness benefit hai jo humne deliberately design kiya.

Q133. Aapne user testing ki hai actual rural entrepreneurs ke saath?

Honest answer do. Agar nahi ki: "Abhi nahi — hackathon timeline mein technical build par focus tha. Design decisions field research aur secondary sources par based hain (high-contrast sunlight-readable UI, voice-first, SMS channel). Next step pilot testing hai 20-30 entrepreneurs ke saath ek SHG ke through, aur usse hum thresholds, prompts aur flow calibrate karenge." Agar ki hai to specific findings batao.

Q134. Is project ka sabse kamzor hissa kya hai?

Do cheezein. Ek — **OCR pipeline**: handwritten Devanagari par accuracy kam hai aur income/expense classification nahi hai; humne ise `confirmed:false` human-in-the-loop se mitigate kiya hai par wo user par bojh daalta hai. Do — **WhatsApp delayed reply**: plan generate hone ke baad usko push karne ka mechanism nahi hai, job queue chahiye. Dono documented limitations hain aur next iteration mein address honi chahiye.

Q135. Agar aapko ye project dobara start karna ho to kya alag karoge?

Teen cheezein. Ek — **job queue pehle din se** (Inngest/QStash), taaki WhatsApp delayed replies aur background OCR properly handle ho. Do — **i18n layer engines mein bhi**, taaki scheme matcher ke reasons directly Hindi mein generate hon, LLM translation par depend na karna pade. Teen — **API integration tests shuru se**, kyunki baad mein add karna hamesha mushkil ho jata hai.

I. Quick-Fire Facts (Rattne layak)

Question	Answer
Kitni tables?	8
Kitni API routes?	9 (+2 auth routes)
Kitni schemes seeded?	15
Kitne conversation states?	6 (idle → ... → complete)
Kitne onboarding steps (web)?	8
Kitne voice steps?	9 (auth + 7 data + complete)
Kitne roadmap stages?	5 (fixed)
Kitne scheme match rules?	6 checks
Kitne financial engine functions?	4
Kitne test cases?	~36 (28 financial + 8 scheme)
LLM model?	gemini-2.5-flash
Temperatures?	0.1 parse, 0.2 roadmap, 0.3 rephrase
Risk thresholds?	0.7 (medium), 0.9 (high)
OCR languages?	eng + hin
OCR amount range filter?	0 < amount < 1,00,00,000
Speech recognition lang?	hi-IN
Next.js version?	16.3.4 (App Router, "proxy" not "middleware")
React version?	19.2.8
Money column type?	NUMERIC(12,2)
Primary key type?	UUID (uuid_generate_v4)
Auth flow?	PKCE with cookie-based sessions
Deployment?	Vercel + Supabase
Problem statement?	SIH26091, Ministry of Social Justice & Empowerment

J. Final Demo Checklist

1. **Landing page** — problem statement aur teen channels dikhao (10 sec)
2. **Login/Onboarding** — voice modal kholo, ek sawaal Hindi mein bol kar dikhao (parse response mein `source: 'gemini'` point out karo)
3. **DPDP consent step** — highlight karo ki bina consent submit hi nahi hota
4. **Dashboard** — margin %, risk level, break-even target dikhao; bolo ki ye **deterministic engine** se aaye hain
5. **Yojana Kendra** — ek eligible scheme kholo, uske ✓ reasons dikhao; phir ek ineligible scheme kholo aur ✗ reason dikhao — **yahi explainability hai**
6. **Business Guide** — ek challenge type karke 5-stage roadmap generate karo

7. **Terminal** — `npx vitest run` chalao, saare tests green dikhao
8. **Code walkthrough** — `financialEngine.ts` kholo aur bolo "zero imports, pure functions"
9. **Limitations slide** — 3-4 honest limitations aur unka fix plan batao

Last tip: Agar kisi sawaal ka jawab nahi pata, to bolo — "*Ye maine implement nahi kiya, par approach ye hoti: ...*" Ya "*Ye ek known limitation hai, aur fix hoga ...*". Jhooth bolna ya bluff karna hamesha pakda jata hai. Jo team apni limitations clearly jaanti hai, wo hamesha zyada credible lagti hai.